

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



ĐỒ ÁN TỐT NGHIỆP

Hệ thống Smart Building với Zigbee và BLE

Lê Đức Anh – 20224403

anh.ld224403@sis.hust.edu.vn

Chu Thiên Phú – 20224450

phu.ct224450@sis.hust.edu.vn

Ngành: Kỹ thuật Điện tử – Viễn thông

Chuyên ngành: Hệ thống nhúng thông minh và IoT

Giáo viên hướng dẫn: Phạm Thị Nhẫn

Chữ ký của GVHD

KHOA: (Điền tên khoa)

Hà Nội, 01/2026

LỜI CẢM ƠN

Mặc dù đã cố gắng hoàn thiện đồ án một cách tốt nhất, song do kiến thức và kinh nghiệm còn hạn chế nên không tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ các thầy cô để đồ án được hoàn thiện hơn.

Hà Nội, tháng <...> năm <...>

Nhóm sinh viên thực hiện

MỤC LỤC

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT	3
DANH SÁCH HÌNH VẼ	5
DANH SÁCH BẢNG	6
PHÂN CHIA CÔNG VIỆC	7
1 TỔNG QUAN ĐỀ TÀI	8
1.1 Bối cảnh và lý do chọn đề tài	8
1.2 Mục tiêu của đề án	9
1.2.1 Mục tiêu tổng quát	9
1.2.2 Mục tiêu cụ thể	9
1.3 Phạm vi và giới hạn	10
1.3.1 Phạm vi nghiên cứu và triển khai	10
1.3.2 Giới hạn của đề tài	10
1.4 Phương pháp thực hiện	10
1.5 Đóng góp chính của đề án	11
1.6 Cấu trúc báo cáo	11
2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN	13
2.1 Tổng quan IoT và Smart Building	13
2.2 Zigbee và IEEE 802.15.4	14
2.3 Vai trò thiết bị trong mạng Zigbee	15
2.4 Topology mạng Zigbee	16
2.5 Zigbee Protocol Stack	17
2.6 Zigbee Cluster Library và mô hình thiết bị	18
2.7 Gateway trong hệ thống IoT	18
2.8 MQTT và cơ chế Publish/Subscribe	19
2.9 UART, ASH và EZSP giữa Host và EFR32 NCP	20
2.10 REST API, Cloud Backend và Database	20
2.11 Mobile App và State Management	21
2.12 Firmware, Bootloader và OTA	21
2.13 Security trong Zigbee và IoT	22
2.14 Công nghệ sử dụng trong đề án	22
3 PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG	24
3.1 Phân tích yêu cầu hệ thống	24
3.1.1 Yêu cầu chức năng	24
3.1.2 Yêu cầu phi chức năng	24
3.2 Kiến trúc tổng thể hệ thống	25
3.3 Thiết kế mạng và Vai trò thiết bị	26
3.4 Luồng dữ liệu chính trong hệ thống	26
3.4.1 Luồng báo cáo trạng thái (Uplink Flow)	26

3.4.2	Luồng gửi lệnh điều khiển (Downlink Flow)	27
3.5	Thiết kế dữ liệu và Hợp đồng giao tiếp (MQTT Contract)	27
3.5.1	Phân cấp cấu trúc Topics	27
3.5.2	Cấu trúc Envelope bản tin chung	28
3.6	Thiết kế cơ sở dữ liệu và Cloud Backend	28
3.6.1	Thiết kế cơ sở dữ liệu quan hệ (PostgreSQL)	28
3.6.2	Thiết kế REST API Endpoints	28
3.7	Các quyết định thiết kế và Trade-offs quan trọng	29
4	TRIỂN KHAI HỆ THỐNG	30
4.1	Triển khai Firmware cho thiết bị Zigbee	30
4.1.1	Triển khai Coordinator / NCP Board	30
4.1.2	Triển khai Router đèn (Z3Light Device)	30
4.1.3	Triển khai Router công tắc (Z3Switch Device)	31
4.1.4	Triển khai cảm biến hiện diện (Occupancy Sensor)	31
4.2	Triển khai Gateway Service nhúng	31
4.2.1	Kiến trúc vận hành C-MQTT trực tiếp	31
4.2.2	Cơ chế ánh xạ Correlation ID	31
4.2.3	Triển khai Rules Engine cục bộ	32
4.3	Triển khai Cloud Backend và quy trình Deployment	32
4.3.1	Cấu hình biến môi trường	32
4.3.2	Triển khai đồng bộ trạng thái và quét Timeout	32
4.3.3	Quy trình Deploy thực tế trên AWS EC2	32
4.4	Luồng vận hành thực tế End-to-End Command	33
4.5	Hướng thiết kế triển khai cập nhật OTA	33
5	KIỂM THỬ VÀ ĐÁNH GIÁ	35
5.1	Kế hoạch kiểm thử	35
5.2	Môi trường và cấu hình kiểm thử	35
5.2.1	Cấu hình phần cứng thử nghiệm	35
5.3	Kết quả thực thi các kịch bản kiểm thử	36
5.3.1	Nhóm 1: Kiểm thử Gia nhập mạng của thiết bị	36
5.3.2	Nhóm 2: Kiểm thử truyền nhận trạng thái và điều khiển	37
5.3.3	Nhóm 3: Kiểm thử tự động hóa cục bộ và độ tin cậy	38
5.4	Đánh giá kết quả kiểm thử	39
5.4.1	Mức độ hoàn thành các tiêu chí hệ thống	39
5.4.2	Phân tích độ trễ của hệ thống	40
5.5	Hạn chế của hệ thống phát hiện qua kiểm thử	40
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	42
6.1	Kết luận chung	42
6.2	Kiến thức và kỹ năng đạt được	42
6.3	Hướng phát triển tiếp theo	43
6.3.1	Hoàn thiện phần cứng và mạng thiết bị	43
6.3.2	Nâng cấp tính năng cho Gateway nhúng	43
6.3.3	Nâng cấp nền tảng Cloud và Ứng dụng di động	43
6.3.4	Tăng cường An ninh bảo mật	44
	TÀI LIỆU THAM KHẢO	45

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Viết tắt	Ý nghĩa
ACK	Acknowledgement
AES	Advanced Encryption Standard
API	Application Programming Interface
APS	Application Support Sub-layer
BLE	Bluetooth Low Energy
CID	Correlation ID
CLI	Command Line Interface
CMD	Command
CMSIS	Cortex Microcontroller Software Interface Standard
CRLF	Carriage Return Line Feed
CSA	Connectivity Standards Alliance
CSMA/CA	Carrier Sense Multiple Access with Collision Avoidance
DMA	Direct Memory Access
EZSP	EmberZNet Serial Protocol
FFD	Full Function Device
GBL	Gecko Bootloader
GPIO	General Purpose Input/Output
GSDK	Gecko SDK
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
I2C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IoT	Internet of Things
JSON	JavaScript Object Notation
LCD	Liquid Crystal Display
LED	Light Emitting Diode
LQI	Link Quality Indicator
LWT	Last Will and Testament (MQTT)
MAC	Media Access Control
MCU	Microcontroller Unit
MQTT	Message Queuing Telemetry Transport
NCP	Network Co-Processor
NVM3	Non-Volatile Memory 3rd Generation
NWK	Network Layer
OTA	Over-The-Air (firmware update)
PAN	Personal Area Network
PHY	Physical Layer
PSA	Platform Security Architecture
PTI	Packet Trace Interface
QoS	Quality of Service
RAIL	Radio Abstraction Interface Layer
REST	Representational State Transfer
RF	Radio Frequency
RFD	Reduced Function Device
RPi	Raspberry Pi

Viết tắt	Ý nghĩa
RTOS	Real-Time Operating System
SDK	Software Development Kit
SED	Sleepy End Device
SoC	System on Chip
SPI	Serial Peripheral Interface
SSv5	Simplicity Studio version 5
TCP/IP	Transmission Control Protocol / Internet Protocol
TLS	Transport Layer Security
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
VCOM	Virtual COM Port
WSTK	Wireless Starter Kit
ZC	Zigbee Coordinator
ZCL	Zigbee Cluster Library
ZDO	Zigbee Device Object
ZED	Zigbee End Device
ZR	Zigbee Router

DANH SÁCH HÌNH VẼ

1.1	Quy trình nghiên cứu và triển khai hệ thống	10
2.1	Kiến trúc tổng quát của hệ thống IoT Zigbee Smart Building trong phạm vi đồ án	14
2.2	Minh họa topology dạng mesh trong mạng Zigbee	16
2.3	Kiến trúc tầng giao thức Zigbee	17
2.4	Mô hình broker trung tâm trong MQTT	19
2.5	Minh họa giao tiếp UART giữa hai thành phần phần cứng	20
3.1	Sơ đồ kiến trúc tổng quan hệ thống Smart Building	25
3.2	Sơ đồ luồng dữ liệu Uplink đồng bộ trạng thái	26
3.3	Sơ đồ luồng điều khiển Downlink từ Cloud xuống thiết bị	27

DANH SÁCH BẢNG

2	Bảng phân chia công việc	7
1.1	Đóng góp chính và ý nghĩa kỹ thuật của đề án	11
2.1	Đặc điểm chính của Zigbee trong hệ thống Smart Building	15
2.2	Vai trò thiết bị trong mạng Zigbee	16
2.3	Một số cluster liên quan đến đề tài	18
2.4	Vai trò của các thành phần Cloud trong hệ thống	21
2.5	Tóm tắt công nghệ nền được sử dụng trong đề án	23
3.1	Các yêu cầu chức năng chính của hệ thống	24
3.2	Các module logic chính trong ứng dụng Z3Gateway C	25
3.3	Bảng ánh xạ device type và các cluster ZCL cấu hình	26
3.4	Thiết kế phân lớp các kênh truyền MQTT	27
3.5	Các bảng cơ sở dữ liệu chính của Cloud Backend	28
3.6	Đặc tả các REST API endpoints chính của hệ thống	29
4.1	Các biến môi trường cấu hình chính của Cloud Backend	32
5.1	Cấu hình địa chỉ và vai trò của các thiết bị trong mạng thử nghiệm	36
5.2	Nhóm kịch bản kiểm thử gia nhập mạng (Device Join)	37
5.3	Nhóm kịch bản kiểm thử truyền tin và điều khiển (Uplink/Downlink)	37
5.4	Nhóm kịch bản kiểm thử tự động hóa và xử lý lỗi (Automation & Faults)	38
5.5	Tổng hợp đánh giá mức độ hoàn thành các tiêu chí hệ thống	40

PHÂN CHIA CÔNG VIỆC

STT	Công việc	Sinh viên thực hiện
Firmware – Zigbee Devices		
1	<Firmware Coordinator: tạo mạng, Trust Center, ZCL, UART bridge>	<SV>
2	<Firmware Light Device: On/Off Server, LED indicator, state report>	<SV>
3	<Firmware Switch Device: On/Off Client, binding, button control>	<SV>
4	<Firmware Occupancy Sensor: occupancy detection, ZCL report>	<SV>
Gateway Service (Ubuntu / Raspberry Pi)		
5	<Gateway UART module: kết nối COM, đọc line, reconnect>	<SV>
6	<Gateway MQTT bridge: subscribe cmd, publish state/telemetry/ack>	<SV>
7	<Gateway rules engine, ACK routing, retry/timeout>	<SV>
8	<Gateway Admin API (/health, /status, /config, /rules)>	<SV>
Cloud – Firebase		
9	<Firebase setup: Realtime DB / Firestore schema, Cloud Functions>	<SV>
10	<MQTT – Firebase bridge: device state sync, command relay>	<SV>
11	<Event history, logging, device management trên cloud>	<SV>
Mobile App – Flutter		
12	<Flutter app: device list, light on/off control UI>	<SV>
13	<Flutter app: occupancy state display, realtime update>	<SV>
14	<Flutter app: event history, notification>	<SV>
Tổng hợp & Tài liệu		
15	<Tổng hợp và hoàn thiện báo cáo>	<SV 1, SV 2>

Bảng 2: Bảng phân chia công việc

CHƯƠNG 1

TỔNG QUAN ĐỀ TÀI

Chương này trình bày bối cảnh và lý do lựa chọn đề tài xây dựng hệ thống IoT Zigbee Smart Building. Tiếp theo, chương xác định rõ mục tiêu tổng quát, mục tiêu cụ thể, phạm vi nghiên cứu và các giới hạn thực tế của đề án. Cuối cùng, phương pháp thực hiện, những đóng góp chính về mặt kỹ thuật và cấu trúc tổng thể của báo cáo cũng được khái quát hóa nhằm cung cấp cái nhìn toàn cảnh trước khi đi vào các nội dung kỹ thuật chi tiết.

1.1 Bối cảnh và lý do chọn đề tài

Trong những năm gần đây, sự phát triển mạnh mẽ của công nghệ mạng không dây và Internet vạn vật (Internet of Things - IoT) đã thúc đẩy việc ứng dụng tự động hóa vào quản lý vận hành tòa nhà. Smart Building (tòa nhà thông minh) không còn là khái niệm lý thuyết mà đã trở thành xu hướng tất yếu nhằm tối ưu hóa lượng điện năng tiêu thụ, nâng cao độ an toàn và mang lại trải nghiệm tiện nghi cho người sử dụng. Trong các hệ thống này, nhu cầu giám sát trạng thái môi trường, phát hiện sự hiện diện của con người và điều khiển các thiết bị phụ tải (như đèn chiếu sáng, công tắc) là những bài toán cốt lõi cần giải quyết.

Đối với mạng kết nối nội bộ (local network) trong tòa nhà, việc sử dụng các công nghệ truyền thông không dây phổ biến như Wi-Fi thường bộc lộ những hạn chế lớn về mặt tiêu thụ năng lượng và khả năng chịu tải của điểm truy cập trung tâm khi số lượng thiết bị lên tới hàng trăm node. Do đó, các công thức truyền thông tiết kiệm năng lượng, hỗ trợ cấu trúc liên kết mạng lưới (mesh network) như Zigbee được xem là giải pháp tối ưu. Zigbee hoạt động trên tiêu chuẩn IEEE 802.15.4, cho phép các thiết bị tự động định tuyến gói tin qua nhau, giúp mở rộng phạm vi phủ sóng mà không yêu cầu công suất phát lớn, đồng thời giảm thiểu đáng kể chi phí duy trì nguồn pin cho các cảm biến.

Tuy nhiên, các mạng Zigbee cục bộ không thể tự kết nối trực tiếp với các ứng dụng di động hoặc hệ thống quản lý từ xa qua mạng IP diện rộng. Điều này đòi hỏi sự xuất hiện của một thiết bị trung gian gọi là Gateway. Thiết bị Gateway đóng vai trò làm cầu nối chuyển đổi giữa giao thức Zigbee RF tầm ngắn và mạng IP (chạy các giao thức như MQTT, HTTP) kết nối lên Cloud. Sự phối hợp giữa Gateway và hệ thống đám mây (Cloud Backend) cho phép lưu trữ lịch sử vận hành, xử lý các kịch bản tự động hóa phức tạp và cung cấp các giao diện lập trình ứng dụng (API) cho ứng dụng di động của người dùng cuối.

Hơn nữa, các giải pháp smart building thương mại hiện nay thường là các hệ sinh thái đóng (closed ecosystem) của các nhà sản xuất lớn, gây khó khăn cho việc tích hợp thiết bị của bên thứ ba và hạn chế khả năng tùy biến logic điều khiển. Xuất phát từ nhu cầu nghiên cứu một nền tảng quản lý thiết bị mở, có khả năng làm chủ từ firmware thiết bị, logic xử lý tại gateway cho đến kiến trúc cloud backend, đề tài "**Hệ thống IoT Zigbee Smart Building**" đã được lựa chọn để thực hiện. Đề án tập trung nghiên cứu thiết kế và tích hợp đồng bộ các lớp thiết bị, gateway và dịch vụ đám mây nhằm đảm bảo luồng dữ liệu hai chiều nhanh chóng, tin cậy và có khả năng mở rộng.

1.2 Mục tiêu của đề án

Để giải quyết các bài toán đặt ra trong bối cảnh trên, đề án xác định rõ mục tiêu tổng quát và các mục tiêu cụ thể cần đạt được.

1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề án là thiết kế, chế tạo và tích hợp hoàn chỉnh một hệ thống IoT quản lý Smart Building dựa trên chuẩn truyền thông không dây Zigbee, kết nối qua Gateway C tới hệ thống Cloud Backend, hỗ trợ giám sát trạng thái thiết bị thực tế gần thời gian thực và tự động hóa điều khiển phụ tải.

1.2.2 Mục tiêu cụ thể

Để hiện thực hóa mục tiêu tổng quát, đề án đề ra các mục tiêu cụ thể cho từng thành phần trong hệ thống:

Phía thiết bị cuối (End Devices): Thiết kế và nạp firmware cho các thiết bị Zigbee 3.0 dựa trên vi điều khiển EFR32MG12 bao gồm thiết bị đèn (Z3Light), công tắc (Z3Switch) và cảm biến hiện diện (Occupancy/Motion Sensor) hoạt động ổn định trong mạng mesh Zigbee.

Phía Gateway trung gian: Triển khai ứng dụng Z3Gateway dạng tiến trình đơn (single-process) viết bằng ngôn ngữ C chạy trên Linux host. Tích hợp trực tiếp thư viện kết nối MQTT để giao tiếp trực tiếp với Cloud, loại bỏ hoàn toàn các lớp trung gian trung chuyển IPC phức tạp nhằm giảm độ trễ và tăng tính ổn định.

Phía dịch vụ đám mây (Cloud Backend): Xây dựng Cloud Backend sử dụng framework FastAPI (Python 3.12) kết nối cơ sở dữ liệu PostgreSQL. Hệ thống đám mây phải tiếp nhận các bản tin telemetry báo cáo trạng thái, quản lý vòng đời lệnh điều khiển, lưu trữ lịch sử sự kiện và cung cấp hệ thống REST API chuẩn hóa.

Phía ứng dụng giám sát (Dashboard/App): Thiết kế các giao diện mô phỏng dưới dạng API hoặc màn hình quản lý cho phép theo dõi trực quan danh sách thiết bị, trạng thái vận hành hiện tại và gửi lệnh bật/tắt thiết bị đèn.

Về mặt kiểm thử và đánh giá: Xây dựng bộ kịch bản kiểm thử (test cases) end-to-end hoàn chỉnh nhằm đo đặc độ trễ phản hồi của lệnh điều khiển, kiểm chứng tính chính xác của cơ chế đồng bộ trạng thái và ghi nhận bằng chứng vận hành (logs, traces).

1.3 Phạm vi và giới hạn

Việc xác định phạm vi nghiên cứu và giới hạn thực thi giúp đồ án tập trung vào các vấn đề kỹ thuật trọng tâm, tránh phân tán nguồn lực vào các tính năng thương mại phức tạp.

1.3.1 Phạm vi nghiên cứu và triển khai

Đồ án tập trung thực hiện các nội dung sau:

Thiết lập mạng Zigbee cục bộ: Sử dụng một Coordinator chạy firmware NCP (Network Co-Processor) kết nối qua UART/ASH với Gateway host.

Triển khai kịch bản gia nhập mạng: Thực hiện permit-join, đồng bộ trạng thái thực tế lên cloud (reported state) và truyền đạt lệnh trạng thái mong muốn (desired state) qua giao thức MQTT.

Đồng bộ trạng thái thiết bị và kịch bản tự động hóa cục bộ: Hỗ trợ local automation cơ bản thông qua cơ chế APS Binding trực tiếp giữa công tắc và đèn trong mạng Zigbee mesh.

Theo dõi vòng đời của lệnh điều khiển: Theo dõi chặt chẽ quá trình xử lý lệnh từ Cloud xuống thiết bị qua các pha trạng thái rõ ràng (accepted → queued → sent → executed/failed/-timeout).

Triển khai hệ thống Cloud Backend: Deploy hệ thống trên hạ tầng điện toán đám mây dưới dạng các container Docker được đóng gói và cấu hình hoàn chỉnh.

1.3.2 Giới hạn của đề tài

Do hạn chế về mặt thời gian và trang thiết bị phần cứng, đồ án tồn tại một số giới hạn như sau:

Quy mô thử nghiệm: Do hạn chế về mặt thời gian và trang thiết bị phần cứng, hệ thống chỉ dừng lại ở quy mô thử nghiệm phòng lab (lab-scale) với số lượng thiết bị demo hạn chế (khoảng 3-5 node), chưa được triển khai thực tế trên quy mô tòa nhà nhiều tầng với mật độ cản trở vật lý cao.

Ứng dụng di động: Ứng dụng di động Flutter và cơ chế điều khiển tích hợp qua giao thức Bluetooth Low Energy (BLE) chỉ dừng lại ở mức thiết kế giao diện tĩnh (placeholders) hoặc mô phỏng logic client, chưa được tích hợp kết nối API thực tế chạy trên thiết bị di động thật trong bản v1 này.

Tính năng nâng cao: Các tính năng nâng cao như cơ chế cập nhật firmware từ xa (OTA Update), bảo mật mã hóa đường truyền MQTT qua TLS (port 8883) và hệ thống xác thực người dùng (Authentication API) chỉ được thực hiện ở mức phân tích thiết kế kịch bản lý thuyết (contracts), chưa có mã nguồn triển khai chạy thực tế.

1.4 Phương pháp thực hiện

Để thực hiện đồ án một cách khoa học, quy trình nghiên cứu và phát triển được chia làm 5 bước tuần tự như được mô tả trong Hình 1.1.

Hình 1.1: Quy trình nghiên cứu và triển khai hệ thống

Quy trình thực hiện bao gồm các giai đoạn cụ thể sau:

Giai đoạn 1 – Khảo sát và nghiên cứu công nghệ: Nghiên cứu tài liệu kỹ thuật về chuẩn Zigbee 3.0, thư viện ZCL (Zigbee Cluster Library) của Silicon Labs, giao thức truyền thông

MQTT và kiến trúc API hướng tài nguyên sử dụng REST.

Giai đoạn 2 – Thiết kế hệ thống: Xác định sơ đồ khối kiến trúc hệ thống 4 lớp, thiết kế cấu trúc cơ sở dữ liệu PostgreSQL, xây dựng hợp đồng trao đổi bản tin MQTT (MQTT Contract JSON Schema) và cấu hình các topic phân cấp.

Giai đoạn 3 – Triển khai chi tiết từng module: Lập trình firmware cho các SoC EFR32MG12; viết mã nguồn Z3Gateway bằng ngôn ngữ C; xây dựng Cloud API bằng FastAPI.

Giai đoạn 4 – Tích hợp hệ thống: Kết nối Gateway host với chip NCP qua UART, kết nối Gateway lên Mosquitto Broker chạy trên container Docker, thiết lập liên kết đồng bộ giữa cơ sở dữ liệu và MQTT Client chạy nền trên Cloud.

Giai đoạn 5 – Kiểm thử và đánh giá: Chạy các kịch bản kiểm thử tích hợp tự động bằng pytest cho backend và thực thi kiểm thử thủ công end-to-end trực tiếp trên phần cứng thật để thu thập log vận hành.

1.5 Đóng góp chính của đề án

Kết quả thực hiện đề án mang lại một số đóng góp chính về mặt giải pháp kỹ thuật tự phát triển thay vì sử dụng các framework đóng gói sẵn của hãng. Bảng 1.1 tóm tắt các đóng góp và ý nghĩa thực tế tương ứng của chúng.

Bảng 1.1: Đóng góp chính và ý nghĩa kỹ thuật của đề án

Giải pháp phát triển	Ý nghĩa thực tế và kỹ thuật
Kiến trúc Gateway C đơn tiến trình tích hợp trực tiếp MQTT client	Loại bỏ hoàn toàn tiến trình trung gian Python IPC cũ, giúp giảm thiểu độ trễ truyền dẫn lệnh xuống thiết bị và tiết kiệm tài nguyên CPU/RAM trên thiết bị host nhúng.
Mô hình theo dõi vòng đời lệnh điều khiển (Command Lifecycle)	Cho phép theo dõi tường minh trạng thái xử lý gói tin qua các pha (accepted, queued, sent, executed, failed, timeout), giải quyết bài toán lệnh bị treo hoặc mất gói vô tuyến mà cloud không phát hiện được.
Chuẩn hóa mô hình nhận dạng thiết bị qua device_id logic	Tách biệt định danh cơ sở dữ liệu khỏi địa chỉ phần cứng cố định (EUI-64) hoặc địa chỉ động (16-bit nwk_addr), tăng tính bảo mật và cho phép linh hoạt thay thế phần cứng lỗi.
Bộ kịch bản kiểm thử E2E tích hợp evidence thực tế	Cung cấp phương pháp luận kiểm thử rõ ràng từ mức đơn vị đến tích hợp hệ thống, đi kèm các bằng chứng vận hành (log trace) chứng minh hệ thống hoạt động chính xác.

Các đóng góp trên được minh chứng thông qua kết quả kiểm thử thực tế tại Chương 5, cho thấy tính khả thi và độ tin cậy của giải pháp tự phát triển trong môi trường nghiên cứu phòng thí nghiệm.

1.6 Cấu trúc báo cáo

Nội dung báo cáo đề án tốt nghiệp được tổ chức thành 6 chương chính sau đây:

Chương 1 – Tổng quan đề tài: Giới thiệu bối cảnh, lý do chọn đề tài, xác định mục tiêu, phạm vi nghiên cứu, phương pháp thực hiện và cấu trúc tổng thể của báo cáo.

Chương 2 – Cơ sở lý thuyết và công nghệ nền: Trình bày nền tảng lý thuyết về giao thức mạng Zigbee, thư viện cluster ZCL, giao thức MQTT, giao tiếp nối tiếp UART/ASH và các kit vi điều khiển phần cứng được sử dụng.

Chương 3 – Phân tích yêu cầu và thiết kế hệ thống: Phân tích các yêu cầu chức năng, phi chức năng, xây dựng sơ đồ kiến trúc tổng thể, mô hình dữ liệu, đặc tả giao diện API và hợp đồng trao đổi bản tin MQTT Contract.

Chương 4 – Triển khai hệ thống: Trình bày chi tiết quá trình lập trình firmware thiết bị, viết ứng dụng Gateway C, lập trình Cloud Backend bằng FastAPI và giải quyết các lỗi thực tế khi deploy lên EC2.

Chương 5 – Kiểm thử và đánh giá: Thiết lập kế hoạch kiểm thử, trình bày kết quả chạy các test case cụ thể, ghi nhận các bằng chứng logs/traces và phân tích ưu điểm cũng như hạn chế của hệ thống.

Chương 6 – Kết luận và hướng phát triển: Tổng kết các kết quả đạt được, tự đánh giá năng lực tích lũy của bản thân và đề xuất các hướng nghiên cứu mở rộng trong tương lai như bảo mật TLS, cập nhật OTA và tích hợp WebSocket.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN

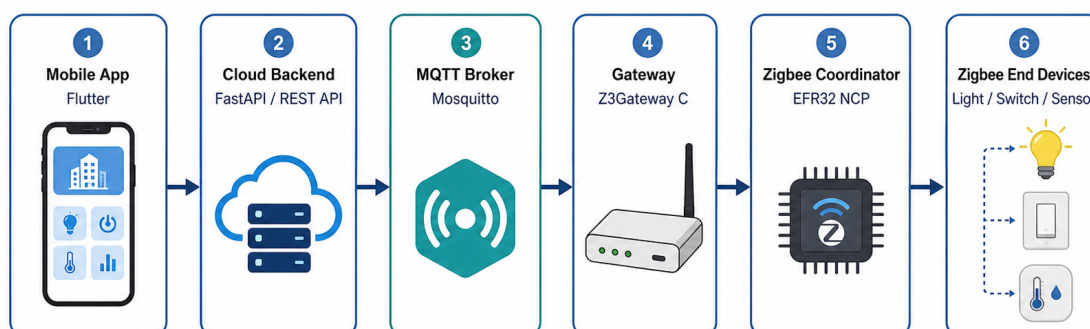
Chương 2 trình bày các cơ sở lý thuyết và công nghệ được sử dụng để xây dựng hệ thống IoT Zigbee Smart Building. Nội dung gồm mô hình IoT trong tòa nhà thông minh, mạng Zigbee, mô hình thiết bị theo Zigbee Cluster Library, vai trò của Gateway, giao thức MQTT, giao tiếp giữa Host và EFR32 NCP, Cloud Backend, Mobile App, firmware/OTA và một số vấn đề bảo mật cơ bản. Các phần dưới đây được trình bày ở mức nền tảng để làm cơ sở cho phần thiết kế và triển khai ở các chương sau, không đi vào log chạy thử hay kết quả thực nghiệm.

2.1 Tổng quan IoT và Smart Building

Internet of Things (IoT), hay Internet vạn vật, là mô hình kết nối các thiết bị vật lý với phần mềm thông qua mạng truyền thông. Thiết bị trong hệ thống IoT thường có cảm biến, bộ xử lý, phần mềm nhúng và khả năng trao đổi dữ liệu với các thành phần khác. Trong phạm vi đề án, IoT được xét theo hướng ứng dụng trong một phòng/lab thông minh, nơi thiết bị có thể gửi trạng thái, nhận lệnh điều khiển và phối hợp với phần mềm phía trên.

Smart Building là một hướng ứng dụng của IoT trong môi trường tòa nhà. Hệ thống thường bao gồm sensor (cảm biến), actuator (thiết bị chấp hành), Gateway (cổng kết nối), Cloud Backend (dịch vụ phía máy chủ) và App (ứng dụng người dùng). Sensor dùng để thu thập trạng thái, ví dụ cảm biến hiện diện phát hiện có người trong phòng. Actuator thực hiện hành động vật lý như bật/tắt đèn hoặc thay đổi trạng thái thiết bị. Gateway nối mạng thiết bị cục bộ với hệ thống phần mềm. Cloud Backend lưu dữ liệu, quản lý thiết bị và cung cấp Application Programming Interface (API) cho App.

Một luồng IoT cơ bản thường bắt đầu từ việc thu thập dữ liệu, truyền dữ liệu, xử lý dữ liệu, phát sinh lệnh điều khiển và hiển thị lại trạng thái cho người dùng. Trong đề án này, người dùng thao tác trên Flutter Mobile App; App gọi FastAPI Cloud REST API; Cloud trao đổi trạng thái và lệnh qua Mosquitto MQTT Broker; Z3Gateway C nhận bản tin MQTT và chuyển thành Zigbee command qua EFR32 NCP; thiết bị Zigbee phản hồi trạng thái theo chiều ngược lại.



Hình 2.1: Kiến trúc tổng quát của hệ thống IoT Zigbee Smart Building trong phạm vi đề án

Cách tổ chức trên giúp tách rõ trách nhiệm giữa các lớp. Mạng Zigbee xử lý truyền thông cục bộ cho thiết bị công suất thấp. Gateway đảm nhiệm chuyển đổi giữa Zigbee và MQTT. Cloud lưu dữ liệu và cung cấp API. App tập trung vào giao diện giám sát và điều khiển. Khi thiết kế hệ thống Smart Building, nhóm thực hiện cân cân bằng giữa xử lý cục bộ và xử lý trên Cloud. Xử lý cục bộ giúp giảm độ trễ trong một số tình huống, còn Cloud thuận lợi hơn cho lưu trữ, quản lý từ xa và mở rộng dữ liệu.

2.2 Zigbee và IEEE 802.15.4

Zigbee là giao thức truyền thông không dây công suất thấp, thường được dùng trong các hệ thống cảm biến và điều khiển [1]. Đặc điểm của Zigbee là tốc độ dữ liệu không cao nhưng tiêu thụ năng lượng thấp, hỗ trợ nhiều thiết bị và có khả năng tổ chức mạng linh hoạt. Các đặc điểm này phù hợp với Smart Building, vì các thiết bị như đèn, công tắc và cảm biến hiện diện chủ yếu trao đổi trạng thái hoặc lệnh ngắn, không truyền dữ liệu dung lượng lớn.

IEEE 802.15.4 là chuẩn ở tầng vật lý và tầng Medium Access Control (MAC) cho mạng cá nhân không dây công suất thấp. Zigbee sử dụng nền tảng này và bổ sung thêm các tầng phía trên như network layer, security và application layer [3]. Có thể hiểu IEEE 802.15.4 quy định cách truyền nhận khung dữ liệu ở mức radio, còn Zigbee quy định thêm cách thiết bị tạo mạng, tham gia mạng, định tuyến và biểu diễn chức năng ứng dụng.

Trong một hệ thống IoT, Zigbee không thay thế hoàn toàn Wi-Fi hoặc Ethernet. Zigbee phù hợp hơn ở lớp thiết bị cục bộ, nơi yêu cầu tiết kiệm năng lượng và truyền các bản tin nhỏ. Đối lại, thiết bị Zigbee thường không giao tiếp trực tiếp với Internet, nên hệ thống cần Gateway làm cầu nối lên Cloud hoặc App.

Bảng 2.1: Đặc điểm chính của Zigbee trong hệ thống Smart Building

Đặc điểm	Ý nghĩa trong hệ thống
Công suất thấp	Phù hợp với cảm biến, công tắc và thiết bị nhúng cần hoạt động lâu dài.
Băng thông thấp	Không phù hợp truyền dữ liệu lớn, nhưng đủ cho trạng thái, sự kiện và lệnh điều khiển.
Hỗ trợ mesh	Có thể mở rộng vùng phủ thông qua các thiết bị có khả năng định tuyến.
Mô hình ZCL	Chuẩn hóa chức năng thiết bị thành cluster, attribute và command.
Cần Gateway	Thiết bị Zigbee không giao tiếp trực tiếp với App/Cloud qua Internet, nên cần Gateway làm cầu nối.

Trong đồ án, Zigbee được dùng cho lớp thiết bị cục bộ. EFR32 NCP đóng vai trò radio coordinator, còn các thiết bị như Light, Switch và Occupancy Sensor là các thiết bị Zigbee trong mạng. Nhờ đó, phần thiết bị nhúng được tách khỏi phần Cloud/App, đồng thời vẫn có thể đồng bộ trạng thái và nhận lệnh điều khiển thông qua Gateway.

2.3 Vai trò thiết bị trong mạng Zigbee

Mạng Zigbee phân chia thiết bị theo vai trò để mạng có thể hình thành, duy trì kết nối và truyền dữ liệu. Ba vai trò cơ bản gồm Coordinator, Router và End Device. Coordinator tạo và

quản lý mạng. Router có thể chuyển tiếp gói tin để mở rộng vùng phủ. End Device là thiết bị cuối, thực hiện chức năng ứng dụng như đo trạng thái hoặc nhận lệnh điều khiển.

Coordinator là thiết bị khởi tạo mạng Zigbee. Thiết bị này chọn kênh, thiết lập Personal Area Network (PAN), quản lý quá trình cho phép thiết bị khác tham gia mạng và thường liên quan đến Trust Center trong cơ chế bảo mật. Trong kiến trúc Host-NCP, vai trò coordinator radio có thể đặt trên EFR32 NCP, còn ứng dụng Gateway chạy ở phía Host.

Router là thiết bị có khả năng chuyển tiếp gói tin cho các node khác. Trong môi trường tòa nhà, Router giúp mở rộng vùng phủ khi thiết bị không thể liên lạc trực tiếp với Coordinator. Tuy nhiên, Router thường cần nguồn cấp ổn định hơn so với End Device vì phải duy trì chức năng định tuyến.

End Device là thiết bị cuối trong mạng. Một End Device có thể là cảm biến hiện diện, công tắc hoặc đèn. Thiết bị này tập trung vào chức năng ứng dụng, ví dụ gửi trạng thái có người/không có người hoặc nhận lệnh bật/tắt. Với các thiết bị dùng pin, End Device có thể được tối ưu để tiết kiệm năng lượng bằng cách ngủ trong một số khoảng thời gian.

Bảng 2.2: Vai trò thiết bị trong mạng Zigbee

Vai trò	Nhiệm vụ chính	Ví dụ trong đề tài
Coordinator	Tạo mạng, quản lý mạng và hỗ trợ thiết bị tham gia mạng.	EFR32 NCP gắn với Gateway.
Router	Chuyển tiếp gói tin, mở rộng vùng phủ mạng.	Thiết bị luôn cấp nguồn nếu được cấu hình hỗ trợ routing.
End Device	Thực hiện chức năng ứng dụng, gửi trạng thái hoặc nhận lệnh.	Light, Switch, Occupancy Sensor.

Phần này không đi sâu vào thuật toán định tuyến của Zigbee. Mục tiêu là làm rõ vì sao Gateway cần một coordinator radio để quản lý mạng, và vì sao các thiết bị như light, switch, sensor có thể được tổ chức như các node trong cùng một mạng Zigbee.

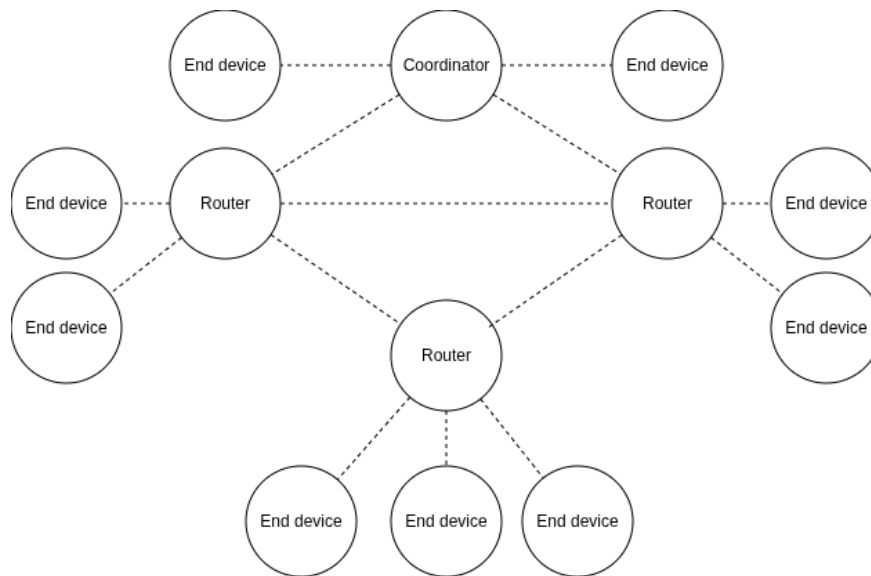
2.4 Topology mạng Zigbee

Topology mô tả cách các node trong mạng được tổ chức và liên kết với nhau. Trong Zigbee, ba dạng topology thường gặp là star, tree và mesh. Việc lựa chọn topology ảnh hưởng trực tiếp đến vùng phủ, độ phức tạp khi quản lý mạng và khả năng mở rộng của hệ thống.

Với star topology, các thiết bị trao đổi dữ liệu thông qua một node trung tâm. Mô hình này đơn giản, dễ kiểm soát và phù hợp với phạm vi nhỏ. Hạn chế của nó là vùng phủ phụ thuộc nhiều vào node trung tâm; nếu thiết bị ở xa hoặc bị vật cản, chất lượng kết nối có thể giảm.

Tree topology tổ chức mạng theo dạng phân cấp. Cách tổ chức này phù hợp khi muốn quản lý theo nhánh, ví dụ theo tầng hoặc theo khu vực trong tòa nhà. Hạn chế là các node phía dưới phụ thuộc vào node trung gian trong nhánh; khi node trung gian gặp sự cố, các node phía sau có thể bị ảnh hưởng.

Mesh topology cho phép một số node chuyển tiếp gói tin qua nhiều đường khác nhau. Đây là topology đáng chú ý trong Smart Building vì môi trường tòa nhà thường có tường, vật cản và khoảng cách giữa các thiết bị. Mesh giúp mở rộng vùng phủ tốt hơn star topology, nhưng việc quản lý mạng và debug cũng phức tạp hơn.



Hình 2.2: Minh họa topology dạng mesh trong mạng Zigbee

Trong đồ án, Zigbee được lựa chọn để tạo mạng cục bộ cho các thiết bị trong phòng/lab. Dù số lượng thiết bị ở giai đoạn đồ án chưa lớn, nền tảng mesh của Zigbee vẫn có ý nghĩa vì nó cho phép hệ thống mở rộng sang nhiều node hơn khi cần.

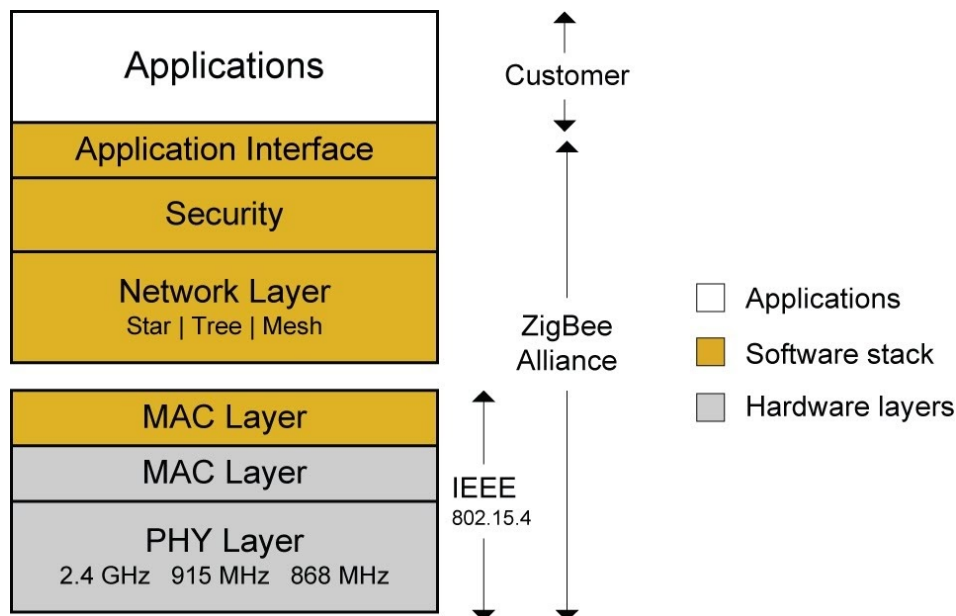
2.5 Zigbee Protocol Stack

Protocol Stack là tập hợp các tầng giao thức cùng tham gia vào quá trình truyền thông. Mỗi tầng đảm nhiệm một nhóm chức năng riêng, từ truyền tín hiệu vô tuyến ở tầng thấp đến biểu diễn lệnh ứng dụng ở tầng cao. Cách phân tầng này giúp hệ thống mạng dễ chuẩn hóa hơn, đồng thời giúp người phát triển xác định lỗi đang nằm ở tầng truyền dẫn, tầng mạng hay tầng ứng dụng.

Tầng Physical (PHY) và MAC của Zigbee dựa trên IEEE 802.15.4. Tầng PHY liên quan đến tần số, kênh truyền và việc phát/thu tín hiệu vô tuyến. Tầng MAC quản lý cách các node truy cập môi trường truyền, định dạng khung dữ liệu cơ bản và một số cơ chế trao đổi ở mức liên kết.

Tầng Network xử lý địa chỉ mạng, quá trình hình thành mạng, quá trình tham gia mạng và định tuyến. Khi một thiết bị mới join mạng, tầng này hỗ trợ cấp địa chỉ và xác định cách thiết bị gửi dữ liệu đến node đích. Với topology mesh, tầng Network có vai trò đáng kể vì gói tin có thể đi qua node trung gian.

Ở phía trên, Application Support (APS) và Application/ZCL xử lý truyền thông ở mức ứng dụng. APS hỗ trợ addressing và binding, còn ZCL định nghĩa cluster, attribute và command. Nhờ đó, lệnh bật/tắt đèn trong hệ thống Zigbee được biểu diễn theo mô hình chuẩn, thay vì dùng chuỗi dữ liệu tự do do từng nhóm tự đặt.



Hình 2.3: Kiến trúc tầng giao thức Zigbee

Ưu điểm của Protocol Stack chuẩn là tính tương thích và khả năng mở rộng tốt hơn. Hạn chế là người phát triển phải nắm thêm nhiều tầng giao thức khi debug. Với đồ án này, lợi ích của việc dùng stack chuẩn lớn hơn độ phức tạp phát sinh, vì hệ thống cần làm việc với nhiều loại thiết bị và cần ánh xạ rõ chức năng của từng thiết bị.

2.6 Zigbee Cluster Library và mô hình thiết bị

Zigbee Cluster Library (ZCL) là bộ đặc tả chuẩn hóa cách mô tả chức năng của thiết bị Zigbee [2]. Nếu các tầng thấp của Zigbee quy định cách dữ liệu đi qua mạng, ZCL quy định dữ liệu đó mang ý nghĩa gì ở tầng ứng dụng. Đây là phần cần thiết trong đồ án vì hệ thống sử dụng các thiết bị như Light, Switch và Occupancy Sensor.

Trong ZCL, cluster là nhóm chức năng của thiết bị. Attribute là trạng thái hoặc thuộc tính nằm trong cluster. Command là lệnh được gửi đến thiết bị hoặc sự kiện do thiết bị phát ra. Ba khái niệm này tạo ra cách biểu diễn thống nhất giữa Gateway và thiết bị Zigbee. Ví dụ, lệnh bật/tắt đèn có thể được biểu diễn bằng On/Off Cluster thay vì một chuỗi text tự định nghĩa.

Bảng 2.3: Một số cluster liên quan đến đèn

Cluster	Identifier	Vai trò
On/Off Cluster	0x0006	Biểu diễn chức năng bật/tắt, phù hợp với light hoặc switch.
Level Control Cluster	0x0008	Biểu diễn mức độ, ví dụ độ sáng của đèn nếu thiết bị hỗ trợ.
Occupancy Sensing Cluster	0x0406	Biểu diễn trạng thái phát hiện hiện diện, phù hợp với occupancy sensor.

Trong luồng điều khiển, App tạo yêu cầu ở mức người dùng, Cloud chuyển yêu cầu thành command logic, Gateway ánh xạ command sang action Zigbee, Zigbee stack đóng gói action theo cluster/command và thiết bị thực hiện lệnh. Ở chiều ngược lại, thiết bị gửi trạng thái mới, Gateway nhận báo cáo và publish reported state để Cloud lưu trữ và App hiển thị.

Việc sử dụng ZCL khiến phần phát triển phức tạp hơn so với tự đặt một payload đơn giản. Tuy nhiên, ZCL giúp hệ thống tránh tình trạng mỗi loại thiết bị dùng một cách biểu diễn riêng. Với đồ án này, lợi ích đó thể hiện rõ ở việc Gateway có thể xử lý lệnh theo loại thiết bị và chức năng, ví dụ tách lệnh bật/tắt light khỏi trạng thái phát hiện hiện diện của sensor.

2.7 Gateway trong hệ thống IoT

Gateway là thành phần nối mạng thiết bị cục bộ với hệ thống phần mềm phía trên. Trong hệ thống IoT Zigbee, thiết bị Zigbee không trực tiếp giao tiếp với REST API hoặc App. Gateway vì vậy cần chuyển đổi giữa hai miền: một bên là mạng Zigbee với cluster, attribute và command; bên còn lại là Cloud, MQTT và các dữ liệu ứng dụng.

Trong đồ án này, Gateway được hiểu theo kiến trúc hiện tại: Z3Gateway C Host App kết nối với EFR32 NCP và tích hợp MQTT trực tiếp thông qua libmosquitto. Kiến trúc Python MQTT-to-IPC bridge cũ không được dùng làm mô hình chính cho báo cáo. Luồng đúng của hệ thống là Cloud/App trao đổi qua MQTT Broker; Z3Gateway C nhận command từ MQTT; Gateway gọi Zigbee action qua Ember AF/EZSP; thiết bị Zigbee phản hồi trạng thái; Gateway publish reported state hoặc event trở lại MQTT.

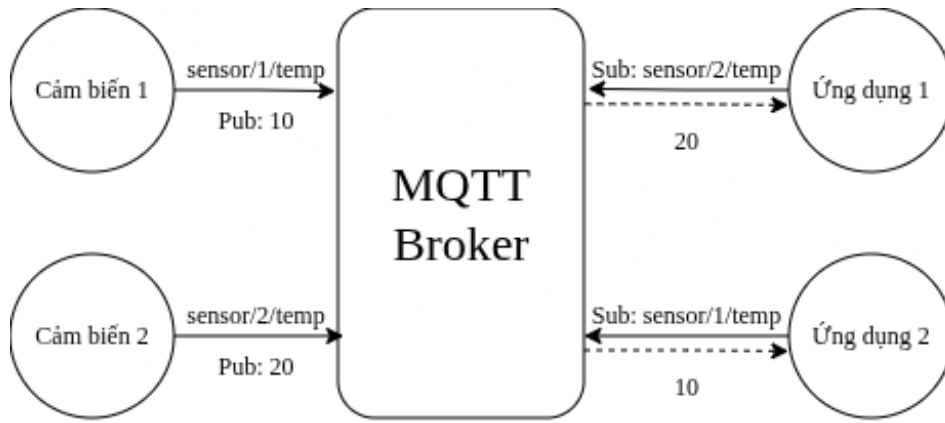
Gateway không chỉ chuyển tiếp dữ liệu thô. Thành phần này còn parse payload, xác định loại thiết bị, ánh xạ command theo device_type, gọi API của Zigbee stack, chuẩn hóa telemetry và publish trạng thái. Trong một số trường hợp, Gateway cũng có thể xử lý local automation ở mức cơ bản, ví dụ sự kiện từ switch hoặc sensor dẫn đến lệnh điều khiển light.

Khi đặt nhiều logic ở Gateway, hệ thống có thể phản hồi nhanh hơn và giảm phụ thuộc vào Cloud trong một số tình huống. Tuy nhiên, Gateway cũng trở thành thành phần khó bảo trì hơn nếu trách nhiệm bị mở rộng quá nhiều. Vì vậy, ranh giới hợp lý là Gateway xử lý kết nối thiết bị và các phản ứng cục bộ cần thiết, còn Cloud đảm nhiệm lưu trữ, API, lịch sử và quản lý người dùng.

2.8 MQTT và cơ chế Publish/Subscribe

Message Queuing Telemetry Transport (MQTT) là giao thức messaging nhẹ, được sử dụng phổ biến trong IoT để truyền telemetry và command giữa các thành phần phân tán [4, 5]. MQTT hoạt động theo mô hình publish/subscribe. Client không gửi dữ liệu trực tiếp cho nhau, mà publish message vào topic; broker nhận message và chuyển tiếp cho các client đang subscribe topic đó.

Các thuật ngữ cơ bản của MQTT gồm broker, client, topic, payload, publish, subscribe, Quality of Service (QoS), retained message và Last Will and Testament (LWT). Broker là trung tâm điều phối message. Client là thành phần kết nối tới broker, chẳng hạn Cloud Backend hoặc Gateway. Topic là đường dẫn logic dùng để phân loại message. Payload là nội dung message. Publish là hành động gửi message, còn subscribe là đăng ký nhận message.



Hình 2.4: Mô hình broker trung tâm trong MQTT

Trong hệ thống của đồ án, Cloud và Gateway cùng kết nối tới Mosquitto MQTT Broker. Ở chiều điều khiển, App gọi REST API lên Cloud, Cloud publish command request xuống topic mà Gateway subscribe. Gateway nhận message, parse payload và chuyển thành Zigbee action. Ở chiều trạng thái, Gateway nhận reported state hoặc event từ mạng Zigbee, sau đó publish lên MQTT để Cloud lưu vào database và App hiển thị lại cho người dùng.

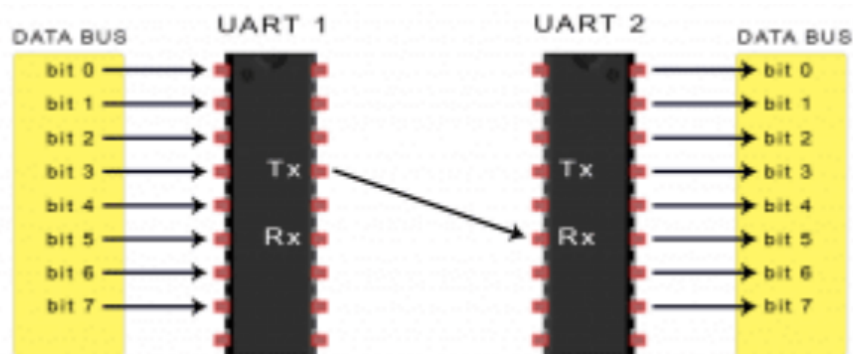
QoS quy định mức đảm bảo truyền message. QoS 0 gửi tối đa một lần, nhẹ nhất nhưng có thể mất message. QoS 1 đảm bảo message đến ít nhất một lần, nhưng bên nhận cần xử lý khả năng trùng lặp. QoS 2 đảm bảo đúng một lần ở mức giao thức, nhưng phức tạp và tốn tài nguyên hơn. Retained message cho phép broker giữ message cuối cùng của một topic, nhờ đó subscriber mới có thể nhận trạng thái gần nhất. LWT giúp broker phát message thay mặt client nếu client mất kết nối bất thường.

MQTT phù hợp với đồ án vì bản tin trạng thái và lệnh điều khiển thường nhỏ, gửi theo sự kiện và cần tách rời Cloud với Gateway. Hạn chế chính là hệ thống phải thiết kế topic và payload rõ ràng. Nếu contract MQTT không ổn định, việc debug và mở rộng hệ thống sẽ khó khăn hơn. Vì vậy, MQTT được xem là contract chính giữa Cloud và Gateway, còn chi tiết topic/payload cụ thể được trình bày ở phần thiết kế và triển khai.

2.9 UART, ASH và EZSP giữa Host và EFR32 NCP

Universal Asynchronous Receiver/Transmitter (UART) là chuẩn giao tiếp nối tiếp bất đồng bộ giữa hai thiết bị [7]. UART thường sử dụng các đường TX, RX và GND; hai bên truyền thông phải thống nhất baud rate, số bit dữ liệu, stop bit và parity. Trong đồ án, UART được xét chủ yếu như đường truyền vật lý/serial giữa Host Gateway và EFR32 NCP, không phải giao thức ứng dụng giữa Gateway và Cloud.

Network Co-Processor (NCP) là mô hình trong đó chip radio xử lý các phần Zigbee network/stack ở mức thấp hơn, còn Host chạy ứng dụng Gateway ở mức cao hơn. Host và NCP cần một cơ chế trao đổi lệnh, sự kiện và dữ liệu. Với nền tảng Silicon Labs, EmberZNet Serial Protocol (EZSP) và Asynchronous Serial Host (ASH) được dùng để hỗ trợ giao tiếp giữa Host và NCP [13, 14].



Hình 2.5: Minh họa giao tiếp UART giữa hai thành phần phần cứng

Điểm cần phân biệt là UART/EZSP/ASH thuộc biên giao tiếp giữa Host và NCP, còn application contract của hệ thống nằm ở MQTT và Zigbee/ZCL action mapping. Do đó, các frame UART tự định nghĩa trong tài liệu cũ không nên được xem là contract production hiện tại của đồ án.

Mô hình Host–NCP giúp nhóm thực hiện tận dụng stack Zigbee và phần radio của Silicon Labs, thay vì phải tự xử lý toàn bộ logic mạng ở mức thấp. Hạn chế là quá trình debug cần xem xét thêm lớp serial, EZSP/ASH và tương tác giữa Host với NCP. Phần này được trình bày ở mức vừa đủ để giải thích vì sao Gateway C có thể điều khiển mạng Zigbee thông qua EFR32 NCP.

2.10 REST API, Cloud Backend và Database

Cloud Backend là lớp phần mềm trung tâm ở phía server. Trong hệ thống Smart Building, Cloud Backend cung cấp REST API cho App, nhận trạng thái từ Gateway, lưu dữ liệu vào database và gửi command xuống Gateway khi người dùng hoặc rule automation yêu cầu. Nhờ có Backend, App không cần làm việc trực tiếp với Zigbee, cluster hoặc NCP.

Representational State Transfer (REST) là phong cách thiết kế API phổ biến trên nền HTTP. App có thể gọi API để lấy danh sách thiết bị, trạng thái hiện tại, lịch sử event hoặc gửi yêu cầu điều khiển. REST dễ triển khai và dễ kiểm thử, phù hợp với các thao tác quản lý thông thường. Hạn chế của REST là mô hình request/response không tối ưu cho cập nhật realtime liên tục nếu dùng một mình; với yêu cầu realtime cao hơn, hệ thống có thể cần thêm WebSocket hoặc cơ chế push.

Database lưu trữ các dữ liệu bền vững của hệ thống. Các nhóm dữ liệu thường gặp gồm device registry, device state, event history, command lifecycle và automation rule. Device registry mô tả thiết bị trong hệ thống. Device state biểu diễn trạng thái hiện tại hoặc lịch sử trạng thái. Event history hỗ trợ truy vết sự kiện. Command lifecycle cho biết lệnh đã được tạo, gửi, xác nhận hay thất bại. Automation rule mô tả điều kiện và hành động tự động.

Trong đồ án, FastAPI được sử dụng cho REST API, SQLAlchemy hỗ trợ Object-Relational Mapping (ORM), PostgreSQL dùng cho dữ liệu production và Paho MQTT giúp Cloud kết nối với broker. Ở Chương 2, nhóm thực hiện chỉ trình bày vai trò của các thành phần này ở mức công nghệ nền; danh sách endpoint, schema chi tiết và xử lý nghiệp vụ được đặt ở các chương sau.

Bảng 2.4: Vai trò của các thành phần Cloud trong hệ thống

Thành phần	Vai trò lý thuyết
REST API	Cung cấp giao diện HTTP để App lấy dữ liệu và gửi yêu cầu điều khiển.
MQTT Client	Subscribe trạng thái từ Gateway và publish command xuống Gateway.
Database	Lưu thiết bị, trạng thái, sự kiện, command và automation rule.
ORM	Ảnh xạ giữa object trong code backend và bảng dữ liệu trong database.

2.11 Mobile App và State Management

Mobile App là lớp tương tác trực tiếp với người dùng cuối. Trong hệ thống Smart Building, người dùng cần xem trạng thái thiết bị, gửi lệnh điều khiển và theo dõi các event hoặc automation rule. App vì vậy chuyển thao tác của người dùng thành request gửi lên Cloud Backend, đồng thời trình bày lại dữ liệu mà Backend cung cấp.

Flutter là framework UI cross-platform, cho phép xây dựng giao diện trên nhiều nền tảng từ cùng một codebase [17]. Trong đồ án, App gọi REST API qua HTTP. Cách tổ chức này giữ App ở lớp người dùng và Cloud, không để App giao tiếp trực tiếp với Zigbee hoặc Gateway. Nhờ đó, App tập trung vào giao diện và trải nghiệm sử dụng, còn Gateway và Cloud xử lý các phần truyền thông thiết bị.

State management là cách App quản lý và cập nhật trạng thái giao diện. Khi danh sách thiết bị, trạng thái đèn hoặc log automation thay đổi, giao diện cần được cập nhật nhất quán. Provider là một cơ chế state management phổ biến trong Flutter, giúp tách dữ liệu/trạng thái khỏi widget hiển thị. Với đồ án này, state management giúp App phản ánh đúng trạng thái thiết bị sau khi người dùng gửi lệnh hoặc khi Gateway báo trạng thái mới.

Ưu điểm của việc để App làm việc qua REST API là kiến trúc rõ ràng và dễ kiểm thử. Hạn chế là cập nhật realtime có thể chưa mượt nếu chỉ dựa vào request/response. Vì vậy, trong phạm vi Chương 2, App được trình bày như lớp monitoring/control cho người dùng; chi tiết màn hình và luồng thao tác được trình bày ở chương thiết kế hoặc triển khai.

2.12 Firmware, Bootloader và OTA

Firmware là phần mềm chạy trực tiếp trên thiết bị nhúng. Khác với ứng dụng mobile hoặc backend chạy trên hệ điều hành tổng quát, firmware làm việc gần hơn với phần cứng như radio, GPIO, cảm biến và bộ nhớ flash. Trong hệ thống Zigbee, firmware quyết định thiết bị hoạt động như light, switch, occupancy sensor hay NCP coordinator.

Bootloader là chương trình nhỏ chạy trước firmware chính, hỗ trợ nạp, kiểm tra hoặc cập nhật firmware [11]. Trong quá trình phát triển, firmware artifact như file .s37 có thể được dùng để flash thiết bị bằng công cụ của Silicon Labs. Artifact không phải là chức năng mà người dùng cuối thao tác trực tiếp, nhưng là đầu ra cần thiết khi build và nạp firmware cho board.

Over-the-Air (OTA) là cơ chế cập nhật firmware qua mạng thay vì flash thủ công qua máy tính [18]. Với Zigbee OTA, hệ thống thường cần OTA image, quản lý version, kiểm tra tính toàn vẹn, OTA server/client và quá trình truyền firmware theo từng phần. OTA có ý nghĩa khi số lượng thiết bị tăng lên hoặc thiết bị được lắp ở vị trí khó tiếp cận.

OTA cũng làm hệ thống phức tạp hơn vì liên quan đến phân phối firmware, xử lý lỗi cập

nhật, kiểm tra version, bảo mật image và khả năng khôi phục khi cập nhật thất bại. Do đó, trong báo cáo này, OTA được trình bày chủ yếu như nền tảng công nghệ và hướng phát triển. Nhóm thực hiện không khẳng định OTA end-to-end đã hoàn chỉnh nếu chưa có bằng chứng triển khai đầy đủ trong các chương sau.

2.13 Security trong Zigbee và IoT

Security trong hệ thống IoT cần được xem xét ở nhiều lớp, từ thiết bị, mạng Zigbee, Gateway, MQTT Broker, Cloud Backend, Database đến App. Với Smart Building, sự cố bảo mật có thể dẫn đến mất dữ liệu hoặc làm sai lệch hành vi điều khiển thiết bị vật lý. Vì vậy, báo cáo cần nêu các nguyên tắc bảo mật cơ bản trước khi đi vào phần thiết kế hệ thống.

Ở lớp Zigbee, các khái niệm cần chú ý gồm network key, application security và Trust Center. Network key giúp bảo vệ truyền thông trong cùng một mạng Zigbee. Trust Center liên quan đến việc quản lý thiết bị tham gia mạng và phân phối khóa trong một số mô hình bảo mật. Khi thiết bị join mạng, hệ thống cần kiểm soát thiết bị nào được phép tham gia và quá trình trao đổi khóa được thực hiện như thế nào.

Ở lớp MQTT và Cloud, broker cần có authentication, phân quyền topic và trong môi trường production nên cân nhắc Transport Layer Security (TLS). Cloud Backend cần kiểm soát quyền truy cập API, kiểm tra dữ liệu đầu vào và bảo vệ database. App cũng cần xử lý token hoặc thông tin đăng nhập cẩn thận để tránh người dùng không hợp lệ gửi command điều khiển thiết bị.

Firmware security là lớp bảo mật quan trọng khác. Firmware signing giúp giảm rủi ro thiết bị nhận firmware giả mạo. Với OTA, yêu cầu bảo mật còn cao hơn vì firmware được truyền qua mạng. Tuy nhiên, các cơ chế như install-code Trust Center join, APS encryption hardening, GBL signing hoặc MQTT mTLS chỉ được xem là nền tảng hoặc hướng phát triển nếu chưa có bằng chứng hoàn thiện trong hệ thống hiện tại.

Việc tăng cường security giúp hệ thống an toàn hơn nhưng cũng làm quá trình triển khai, vận hành và debug phức tạp hơn. Vì vậy, đồ án cần trình bày đúng phạm vi: nêu nguyên tắc bảo mật nền tảng, chỉ khẳng định những phần có evidence, và đưa các cơ chế nâng cao vào hướng phát triển khi chưa triển khai đầy đủ.

2.14 Công nghệ sử dụng trong đồ án

Các công nghệ trong đồ án được tổ chức theo kiến trúc nhiều lớp. Thiết bị Zigbee nằm ở tầng nhúng và mạng cục bộ. Gateway chịu trách nhiệm chuyển đổi giữa Zigbee và MQTT. MQTT Broker làm trung gian messaging. Cloud Backend quản lý dữ liệu và API. Mobile App là giao diện cho người dùng cuối. Cách chia lớp này giúp hệ thống dễ giải thích, dễ kiểm thử theo từng phần và thuận lợi hơn khi mở rộng.

Bảng 2.5: Tóm tắt công nghệ nền được sử dụng trong đồ án

Lớp	Công nghệ	Vai trò trong hệ thống
Device	EFR32, EmberZNet/Gecko SDK, Zigbee stack	Chạy firmware cho light, switch, occupancy sensor hoặc NCP.
Gateway	Z3GatewayHost, C, libmosquitto	Kết nối Zigbee local network với MQTT/Cloud.
Broker	Eclipse Mosquitto	Trung gian publish/subscribe giữa Cloud và Gateway.
Backend	FastAPI, SQLAlchemy, Paho MQTT	Cung cấp REST API, xử lý MQTT và quản lý dữ liệu/lệnh.
Database	PostgreSQL, SQLite trong một số ngữ cảnh test	Lưu thiết bị, trạng thái, sự kiện, command và automation.
App	Flutter, Dart, Provider, HTTP	Giao diện monitoring/control cho người dùng cuối.

Tóm lại, Chương 2 cung cấp phần cơ sở để người đọc hiểu các lựa chọn kỹ thuật chính của đồ án: Zigbee cho mạng thiết bị, Z3Gateway C cho lớp kết nối với coordinator radio, MQTT cho trao đổi trạng thái/lệnh, REST API và database cho lớp Cloud, Flutter cho giao diện người dùng. Các khái niệm này là tiền đề cho phân phân tích yêu cầu, thiết kế kiến trúc, triển khai và đánh giá ở các chương tiếp theo.

CHƯƠNG 3

PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG

Chương này tập trung trình bày các kết quả phân tích yêu cầu chức năng, phi chức năng và thiết kế kiến trúc cho toàn bộ hệ thống Smart Building Platform. Nội dung chi tiết bao gồm mô hình kiến trúc phân lớp, các luồng dữ liệu Uplink/Downlink, thiết kế mô hình dữ liệu thiết bị, thiết kế cấu trúc cơ sở dữ liệu, đặc tả giao diện REST API và đặc tả hợp đồng bản tin MQTT Contract, cùng phân tích các quyết định trade-off thiết kế quan trọng.

3.1 Phân tích yêu cầu hệ thống

Để xây dựng một hệ thống IoT hoàn chỉnh hoạt động ổn định và có thể mở rộng, trước tiên cần phân tích các yêu cầu chức năng và phi chức năng cốt lõi.

3.1.1 Yêu cầu chức năng

Hệ thống được thiết kế để đáp ứng các yêu cầu chức năng chính được phân lớp rõ ràng, tóm tắt trong Bảng 3.1.

Bảng 3.1: Các yêu cầu chức năng chính của hệ thống

Nhóm tính năng	Yêu cầu chức năng	Mô tả chi tiết
Quản lý thiết bị	Gia nhập mạng Zigbee	Cho phép thiết bị Zigbee mới gia nhập vào mạng thông qua việc kích hoạt mở permit-join từ xa.
	Quản lý danh mục	Quản lý thông tin tổ chức tòa nhà (nhà, phòng) và danh sách thiết bị logic qua REST API.
Giám sát & Điều khiển	Báo cáo trạng thái	Cập nhật và lưu trữ trạng thái thực tế (reported state) của thiết bị lên Cloud khi có thay đổi.
	Điều khiển phụ tải	Cho phép gửi lệnh bật/tắt thiết bị đèn từ Cloud xuống mạng Zigbee thông qua Gateway.
Tự động hóa	Liên kết trực tiếp	Cho phép công tắc điều khiển đèn cục bộ thông qua cơ chế binding lớp vô tuyến của Zigbee.
	Theo dõi vòng đời lệnh	Theo dõi trạng thái lệnh qua các pha (accepted, queued, sent, executed, failed, timeout).

Chi tiết các yêu cầu chức năng trong Bảng 3.1 làm định hướng để thiết kế cấu trúc cơ sở dữ liệu và các API endpoints tương ứng tại các mục tiếp theo.

3.1.2 Yêu cầu phi chức năng

Hệ thống phải đảm bảo các tiêu chí kỹ thuật vận hành phi chức năng sau:

Độ trễ phản hồi (Latency): Thời gian xử lý lệnh từ lúc client gọi API gửi lệnh điều khiển cho tới khi nhận được phản hồi trạng thái từ thiết bị đèn thực tế qua Gateway phải nằm trong ngưỡng cho phép (dưới 2 giây trong điều kiện mạng ổn định).

Khả năng hoạt động độc lập (Offline Capability): Khi mất kết nối internet lên Cloud Backend, các kịch bản tự động hóa cục bộ (công tắc bật/tắt đèn qua local binding) phải hoạt động bình thường nhờ logic xử lý trọn vẹn trong mạng mesh Zigbee.

Giới hạn thời gian (Timeout): Thời gian chờ phản hồi tối đa của lệnh điều khiển thiết bị được thiết lập cố định là 5000 ms để tránh tình trạng lệnh bị treo vô hạn trong cơ sở dữ liệu.

Bảo mật phân quyền (Least Privilege): Broker Mosquitto phải cấu hình kiểm soát truy cập (ACL) phân tách rõ ràng quyền đọc/ghi của các tài khoản (gateway, client, monitor) để tránh lộ thông tin và can thiệp lệnh trái phép.

3.2 Kiến trúc tổng thể hệ thống

Hệ thống được thiết kế theo mô hình kiến trúc phân lớp hướng dịch vụ để phân tách rõ ràng trách nhiệm của từng thành phần. Hình 3.1 mô tả cấu trúc 4 lớp chính của hệ thống.

Hình 3.1: Sơ đồ kiến trúc tổng quan hệ thống Smart Building

Kiến trúc hệ thống bao gồm bốn lớp vật lý chính phối hợp hoạt động:

Lớp Thiết bị (Device Layer): Gồm các nút thiết bị Zigbee 3.0 (Z3Light, Z3Switch, Occupancy Sensor) giao tiếp không dây với Coordinator.

Lớp Gateway trung gian (Gateway Layer): Gồm phần cứng host nhúng chạy hệ điều hành Linux và ứng dụng native C Z3Gateway. Ứng dụng này sử dụng chip NCP qua UART/ASH để điều khiển mạng Zigbee, đồng thời kết nối trực tiếp lên MQTT Broker. Bảng 3.2 liệt kê vai trò của các module mã nguồn tự thiết kế trong Z3Gateway.

Lớp Trung chuyển (Message Broker): Mosquitto MQTT Broker làm trung gian điều phối bản tin, hỗ trợ ACL bảo mật và cơ chế cầu nối (bridge) truyền thông.

Lớp Dịch vụ đám mây (Cloud Layer): Chứa dịch vụ Cloud Backend xây dựng trên framework FastAPI kết nối cơ sở dữ liệu PostgreSQL để lưu trữ trạng thái lâu bền và expose hệ thống REST API phục vụ người dùng.

Bảng 3.2: Các module logic chính trong ứng dụng Z3Gateway C

Tên module	Vai trò và nhiệm vụ kỹ thuật
app_mqtt.c/h	Khởi tạo client MQTT kết nối Broker, xử lý đăng ký nhận và phát bản tin.
cmd_handler.c/h	Phân tích cấu trúc envelope JSON của lệnh, dispatch lệnh tới module phần cứng.
device_registry.c/h	Quản lý bảng lưu trữ thông tin thiết bị trong bộ nhớ RAM tạm thời (eui64, 16-bit nwk_addr).
light_ctrl.c/h	Tạo các khung tin nhắn ZCL On/Off gửi xuống Coordinator để bật/tắt đèn.
telemetry_rx.c/h	Bắt các callback báo cáo thuộc tính ZCL từ NCP, chuyển đổi thành JSON reported.
net_mgr.c/h	Quản lý quá trình khởi tạo mạng Zigbee và mở cửa sổ cho phép thiết bị join.

Sự phân chia rõ ràng các module trong Bảng 3.2 đảm bảo tính mô-đun hóa cao, giúp Z3Gateway chạy ổn định dưới dạng một single-process C gọn nhẹ.

3.3 Thiết kế mạng và Vai trò thiết bị

Mạng Zigbee cục bộ sử dụng mô hình mesh-tree phân cấp. Coordinator đóng vai trò Trust Center duy nhất để quản lý bảo mật mạng, các node Z3Light và Z3Switch hoạt động dưới vai trò Router giúp chuyển tiếp gói tin, cảm biến chuyển động hoạt động dưới vai trò End Device tiết kiệm pin. Bảng 3.3 mô tả chi tiết sự ánh xạ giữa loại thiết bị và các cluster ZCL tương ứng được cấu hình.

Bảng 3.3: Bảng ánh xạ device type và các cluster ZCL cấu hình

Loại thiết bị	Zigbee Role	Server Clusters	Client Clusters
Coordinator / NCP	ZC (Coordinator)	–	On/Off (0x0006)
Light Device	ZR (Router)	On/Off (0x0006)	–
Switch Device	ZR (Router)	–	On/Off (0x0006)
Occupancy Sensor	ZED (End Device)	Occupancy Sensing (0x0406)	–

Cấu hình cluster trong Bảng 3.3 đảm bảo các thiết bị chỉ thực hiện các chức năng tối thiểu theo đúng thiết kế của Silicon Labs để tiết kiệm bộ nhớ flash của chip.

3.4 Luồng dữ liệu chính trong hệ thống

Hệ thống được thiết kế dựa trên hai luồng truyền dẫn dữ liệu hai chiều Uplink và Downlink cốt lõi, cùng luồng tự động hóa cục bộ độc lập.

3.4.1 Luồng báo cáo trạng thái (Uplink Flow)

Uplink flow đảm nhận việc đồng bộ trạng thái thực tế của thiết bị vật lý lên cơ sở dữ liệu Cloud gần thời gian thực. Hình 3.2 mô tả chi tiết đường đi của dòng dữ liệu từ cảm biến lên giao diện giám sát.

Hình 3.2: Sơ đồ luồng dữ liệu Uplink đồng bộ trạng thái

Quá trình truyền dẫn Uplink diễn ra theo các bước tuần tự sau:

Bước 1 – Gửi báo cáo từ thiết bị: Thiết bị Zigbee (Light/Sensor) gửi bản tin báo cáo thuộc tính ZCL Attribute Report lên Coordinator.

Bước 2 – Nhận và chuyển đổi tại Coordinator: Coordinator nhận được gói tin vô tuyến, chuyển đổi thành khung tin nhắn EZSP truyền qua UART/ASH nối tiếp lên Host Gateway.

Bước 3 – Xử lý và publish tại Gateway: Module `telemetry_rx.c` trên Z3Gateway bắt phản hồi dữ liệu, phân tích giá trị nhị phân thành định dạng trạng thái tương ứng, sau đó chuyển cho module `app_mqtt.c` để xuất bản tin nhắn được báo cáo lên MQTT Broker.

Bước 4 – Đăng ký và lưu trữ tại Cloud: Cloud Backend đăng kí chủ đề được báo cáo qua MQTT client chạy nền, bắt được tin nhắn và lưu trữ giá trị trạng thái mới vào PostgreSQL.

3.4.2 Luồng gửi lệnh điều khiển (Downlink Flow)

Downlink flow chịu trách nhiệm truyền tải lệnh điều khiển bật/tắt từ người dùng xuống thiết bị chấp hành cuối. Hình 3.3 thể hiện chi tiết quá trình này.

Hình 3.3: Sơ đồ luồng điều khiển Downlink từ Cloud xuống thiết bị

Các bước cụ thể trong quy trình Downlink bao gồm:

Bước 1 – Gửi yêu cầu điều khiển từ UI: Người dùng thao tác trên giao diện, gọi REST API điều khiển (POST /api/devices/{id}/command).

Bước 2 – Tiếp nhận và gửi yêu cầu từ Cloud: Cloud API tạo bản ghi lệnh điều khiển mới trong cơ sở dữ liệu với trạng thái khởi tạo accepted, đồng thời xuất bản tin nhắn lệnh điều khiển lên MQTT topic của Gateway.

Bước 3 – Nhận và phân tích lệnh tại Gateway: Gateway nhận được bản tin MQTT qua module `app_mqtt.c`, tiến hành phân tích JSON envelope tại `cmd_handler.c` để xác minh tính hợp lệ.

Bước 4 – Tra cứu địa chỉ và gửi lệnh ZCL: Gateway tra cứu bảng địa chỉ thiết bị trong RAM tại `device_registry.c` để lấy địa chỉ 16-bit tương ứng của Light, sau đó gọi `light_ctrl.c` gửi khung lệnh ZCL On/Off xuống Coordinator qua UART.

Bước 5 – Thực thi lệnh và phản hồi: Thiết bị Light thực thi lệnh vật lý, đổi trạng thái LED và gửi trả bản tin phản hồi ngược lại Gateway để báo cáo lệnh đã chạy thành công.

3.5 Thiết kế dữ liệu và Hợp đồng giao tiếp (MQTT Contract)

Để đảm bảo các tiến trình viết bằng các ngôn ngữ khác nhau (C trên Gateway, Python trên Cloud) có thể trao đổi thông tin chính xác, hệ thống định nghĩa một bộ hợp đồng giao tiếp chuẩn hóa duy nhất dựa trên JSON Schema.

3.5.1 Phân cấp cấu trúc Topics

Hệ thống sử dụng cấu trúc topic phân cấp hỗ trợ đa khách hàng (multi-tenant) và đa địa điểm (multi-site):

```
sb/v1/{tenant_id}/{site_id}/{gateway_id}/{channel_type}/...
```

Trong đó, các giá trị mặc định được cấu hình trong môi trường thử nghiệm bao gồm: `tenant_id = hust`, `site_id = lab01`, và `gateway_id = gw-ubuntu-01`. Bảng 3.4 mô tả các nhóm kênh truyền MQTT được thiết kế.

Bảng 3.4: Thiết kế phân lớp các kênh truyền MQTT

Nhóm kênh	Hướng truyền	Publisher	Subscriber	QoS
gateway/online	Uplink	Gateway	Cloud	1 (Retain)
gateway/health	Uplink	Gateway	Cloud	0
reported	Uplink	Gateway	Cloud	1 (Retain)
command	Downlink	Cloud	Gateway	1
reply	Uplink	Gateway	Cloud	1

Việc cấu hình QoS 1 cho các kênh `command` và `reply` trong Bảng 3.4 đảm bảo lệnh điều khiển không bị mất mát khi đường truyền mạng WAN gặp sự cố ngắn hạn.

3.5.2 Cấu trúc Envelope bản tin chung

Mọi tin nhắn trao đổi qua lại giữa Gateway và Cloud đều bắt buộc sử dụng một envelope JSON chung chứa siêu dữ liệu định danh, giúp thống nhất việc deserialization:

Mã nguồn 3.1: Cấu trúc envelope JSON chung của bản tin

```
{
  "schema": "sb.v1",
  "msg_id": "uuid-cua-ban-tin",
  "ts": "ISO8601-timestamp",
  "tenant_id": "hust",
  "site_id": "lab01",
  "gateway_id": "gw-ubuntu-01",
  "source": "gateway|cloud",
  "correlation_id": "uuid-lien-ket-optional",
  "payload": {}
}
```

Trường correlation_id được sử dụng để gắn kết kết quả phản hồi trên kênh reply với mã lệnh điều khiển gốc gửi đi từ kênh command, giúp Cloud theo dõi chính xác tiến trình xử lý lệnh.

3.6 Thiết kế cơ sở dữ liệu và Cloud Backend

Dịch vụ đám mây xử lý lưu trữ trạng thái lâu bền và expose giao diện REST API.

3.6.1 Thiết kế cơ sở dữ liệu quan hệ (PostgreSQL)

Cấu trúc cơ sở dữ liệu quan hệ PostgreSQL gồm 7 bảng dữ liệu chính để quản lý tổ chức hệ thống, được tóm tắt trong Bảng 3.5.

Bảng 3.5: Các bảng cơ sở dữ liệu chính của Cloud Backend

Tên bảng	Vai trò nhiệm vụ lưu trữ
homes	Lưu thông tin tòa nhà/khu vực lớn nhất.
rooms	Lưu thông tin phân chia các phòng thuộc tòa nhà.
users	Quản lý thông tin tài khoản người dùng hệ thống.
devices	Quản lý danh mục thiết bị logic (lưu device_id logic và EUI-64).
device_states	Lưu trữ ảnh trạng thái hiện tại (reported state) dạng JSON.
events	Lưu log lịch sử các sự kiện xảy ra trong hệ thống.
commands	Theo dõi trạng thái chi tiết của lệnh điều khiển trong vòng đời.

3.6.2 Thiết kế REST API Endpoints

Hệ thống Cloud Backend expose các endpoints API theo tiêu chuẩn RESTful, sử dụng định dạng JSON làm chuẩn trao đổi thông tin. Bảng 3.6 liệt kê các REST API cốt lõi được thiết kế phục vụ ứng dụng.

Bảng 3.6: Đặc tả các REST API endpoints chính của hệ thống

Method	Path	Ý nghĩa chức năng
GET	/health	Kiểm tra trạng thái hoạt động của Cloud API.
GET	/api/devices	Liệt kê danh sách các thiết bị trong hệ thống.
GET	/api/devices/{device_id}/state	Truy vấn trạng thái reported mới nhất của thiết bị.
POST	/api/devices/{device_id}/command	Tạo lệnh điều khiển mới, gửi command xuống Gateway.
GET	/api/commands/{command_id}	Tra cứu trạng thái xử lý hiện tại của lệnh điều khiển.
GET	/api/events	Truy vấn lịch sử sự kiện (phân trang và lọc theo thiết bị).

3.7 Các quyết định thiết kế và Trade-offs quan trọng

Trong quá trình thiết kế hệ thống, một số quyết định kỹ thuật quan trọng đã được đưa ra để tối ưu hiệu năng:

Loại bỏ hoàn toàn cầu nối Python trung gian tại Gateway (Tháng 4/2026): Kiến trúc cũ sử dụng một script Python chạy nền để làm cầu nối chuyển đổi IPC (Unix socket/NDJSON) sang MQTT. Quyết định tích hợp thư viện MQTT trực tiếp vào mã nguồn C của Z3Gateway giúp loại bỏ một tiến trình, tiết kiệm tài nguyên CPU nhúng và giảm trễ truyền dẫn. Tuy nhiên, đánh đổi lại là việc duy trì và thay đổi cấu trúc dữ liệu trên Gateway C đòi hỏi biên dịch lại mã nguồn C phức tạp hơn.

Sử dụng định danh logic device_id thay vì EUI-64 làm khóa bền: Địa chỉ EUI-64 gắn chặt với chip vô tuyến vật lý. Thiết kế sử dụng device_id logic tự sinh làm khóa chính trên Cloud giúp hệ thống dễ dàng thay thế phần cứng thiết bị bị hỏng bằng một thiết bị mới mà không cần cập nhật lại toàn bộ các liên kết trong database.

Chỉ giữ lại bản tin State, không giữ lại bản tin Command: Việc giữ lại bản tin trạng thái reported giúp các client mobile khi kết nối sau lấy ngay được giao diện cập nhật. Đối với command, không sử dụng retain để tránh tình trạng khi Gateway mất kết nối và online trở lại bị Broker gửi lại các lệnh bật/tắt cũ, gây nguy hiểm cho thiết bị chấp hành thực tế.

CHƯƠNG 4

TRIỂN KHAI HỆ THỐNG

Chương này mô tả chi tiết quá trình lập trình triển khai mã nguồn thực tế cho các thành phần trong hệ thống. Nội dung bao gồm triển khai firmware thiết bị Zigbee trên kit EFR32MG12, lập trình Gateway Service C tích hợp MQTT trực tiếp, cấu hình và triển khai Cloud API sử dụng FastAPI đóng gói Docker trên AWS EC2, giải quyết các lỗi triển khai thực tiễn, phân tích hoạt động của luồng điều khiển end-to-end và đề xuất thiết kế hướng phát triển tính năng OTA.

4.1 Triển khai Firmware cho thiết bị Zigbee

Firmware cho các node thiết bị Zigbee 3.0 được lập trình bằng ngôn ngữ C sử dụng IDE Simplicity Studio phiên bản 5 kết hợp ngăn xếp EmberZNet thuộc Gecko SDK phiên bản 4.5.0.

4.1.1 Triển khai Coordinator / NCP Board

Firmware NCP (Network Co-Processor) được biên dịch cho kit BRD4162A đóng vai trò Coordinator. Firmware này không chứa logic ứng dụng smart home mà chỉ hoạt động như một card vô tuyến RF phối bày các dịch vụ EZSP qua UART. Khi khởi động, firmware khởi tạo lớp PHY/MAC trên dải tần 2.4 GHz, thiết lập Trust Center phục vụ trao đổi mã khóa an toàn và liên tục lắng nghe/gửi các gói tin Zigbee theo lệnh điều khiển từ Host Gateway nhúng truyền xuống qua UART.

4.1.2 Triển khai Router đèn (Z3Light Device)

Thiết bị đèn thông minh Z3Light được cấu hình hoạt động ở vai trò Router vô tuyến Zigbee, implement cụm chức năng On/Off Server Cluster (0x0006). Khi nhận được ZCL command yêu cầu bật hoặc tắt, callback của stack EmberZNet sẽ kích hoạt thay đổi mức logic trên chân GPIO điều khiển LED0 trên kit phần cứng:

Mã nguồn 4.1: Snippet xử lý thay đổi trạng thái LED trên Z3Light

```
void emberAfOnOffClusterServerAttributeChangedCallback(uint8_t endpoint,
                                                         EmberAfAttributeId
                                                         attributeId)
{
    if (attributeId == ATTRIBUTE_FREE_ATTRIBUTE_ID) {
        bool onOffState;
        emberAfReadServerAttribute(endpoint,
                                    ZCL_ON_OFF_CLUSTER_ID,
                                    ZCL_ON_OFF_ATTRIBUTE_ID,
                                    (uint8_t *)&onOffState,
                                    sizeof(onOffState));

        if (onOffState) {
            halSetLed(0); // LED0 ON
        }
    }
}
```



```

    } else {
        halClearLed(0); // LED0 OFF
    }
}
}

```

Đồng thời, sau khi thay đổi trạng thái thuộc tính OnOff, firmware tự động kích hoạt gửi bản tin ZCL Attribute Report lên Coordinator để cập nhật trạng thái mới nhất về hệ thống điều khiển trung tâm.

4.1.3 Triển khai Router công tắc (Z3Switch Device)

Thiết bị Z3Switch được cấu hình ở vai trò Router, implement cụm chức năng On/Off Client Cluster. Khi người dùng nhấn nút nhấn BTN0 trên bo mạch, firmware sẽ kích hoạt gửi đi bản tin ZCL Toggle Command trực tiếp tới địa chỉ endpoint đã được lưu trong Bảng Binding cục bộ mà không cần gửi gói tin qua Gateway. BTN1 được cấu hình để kích hoạt tính năng Network Steering giúp thiết bị tự động quét các mạng lân cận để gia nhập mạng.

4.1.4 Triển khai cảm biến hiện diện (Occupancy Sensor)

Cảm biến hiện diện được triển khai ở vai trò Sleepy End Device, sử dụng cụm chức năng Occupancy Sensing Server Cluster (0x0406). Để tiết kiệm năng lượng, firmware đi vào trạng thái ngủ sâu (sleep mode) tắt khối RF. Khi nút nhấn BTN0 được nhấn (mô phỏng cảm biến phát hiện chuyển động), một ngắt ngoài GPIO sẽ đánh thức vi điều khiển. Firmware lập tức thay đổi thuộc tính Occupancy thành True, gửi bản tin Attribute Report lên Parent Node để chuyển về Gateway, sau đó tự động thiết lập một bộ hẹn giờ tự tắt (vacancy timeout) để chuyển thuộc tính về False trước khi quay lại trạng thái ngủ sâu.

4.2 Triển khai Gateway Service nhúng

Ứng dụng Gateway được lập trình bằng ngôn ngữ C, biên dịch chéo cho môi trường Linux sử dụng tệp Makefile tự sinh từ Simplicity Studio.

4.2.1 Kiến trúc vận hành C-MQTT trực tiếp

Z3Gateway hoạt động như một tiến trình đơn (single-process) tích hợp trực tiếp thư viện Eclipse Paho MQTT C. Tiến trình Gateway chạy một vòng lặp sự kiện chính (event loop) để liên tục thăm dò dữ liệu nối tiếp từ cổng UART kết nối với NCP qua giao thức ASH/EZSP, đồng thời lắng nghe các bản tin TCP/IP từ socket kết nối lên MQTT Broker. Việc tích hợp direct MQTT client giúp loại bỏ hoàn toàn cầu nối Python và Unix socket IPC cũ, giải quyết triệt để các lỗi đồng bộ tiến trình và trễ hàng đợi.

4.2.2 Cơ chế ánh xạ Correlation ID

Khi Gateway nhận được một bản tin yêu cầu điều khiển từ kênh command, module cmd_handler.c sẽ bóc tách trường msg_id (UUID tự sinh từ Cloud) lưu vào bộ nhớ RAM tạm thời của Gateway. Khi gửi lệnh điều khiển ZCL xuống thiết bị thành công hoặc thất bại, Gateway sẽ publish bản tin phản hồi lên kênh reply với trường correlation_id được set bằng đúng giá trị msg_id đã lưu trước đó. Cơ chế này giúp Cloud Backend ánh xạ chính xác kết quả thực thi ở

biên vật lý về đúng yêu cầu API gốc của người dùng.

4.2.3 Triển khai Rules Engine cục bộ

Mã nguồn Gateway bao gồm các module `rule_engine.c` và `sb_command.c` chịu trách nhiệm xử lý các kịch bản tự động hóa cục bộ ở biên. Tuy nhiên, trong phiên bản hiện tại v1, logic xử lý của các file này mới dừng lại ở mức thiết lập khung định nghĩa cấu trúc dữ liệu cấu hình rule, việc đánh giá so sánh động các điều kiện rule tự chọn vẫn đang ở giai đoạn phát triển và sẽ được hoàn thiện trong tương lai.

4.3 Triển khai Cloud Backend và quy trình Deployment

Cloud Backend được xây dựng bằng Python 3.12 sử dụng framework FastAPI phiên bản 0.115.6, kết nối PostgreSQL phiên bản 16 qua thư viện SQLAlchemy 2.0.36 hỗ trợ chế độ kết nối bất đồng bộ `asyncpg`.

4.3.1 Cấu hình biến môi trường

Hệ thống Cloud Backend đọc toàn bộ cấu hình vận hành từ biến môi trường thông qua thư viện `Pydantic Settings` với tiền tố `SB_`. Bảng 4.1 liệt kê các biến cấu hình hệ thống chính.

Bảng 4.1: Các biến môi trường cấu hình chính của Cloud Backend

Biến cấu hình	Giá trị mặc định / Ý nghĩa
SB_DATABASE_URL	postgresql+asyncpg://sb_user:sb_pass@localhost:5432/sb_cloud
SB_MQTT_HOST	Địa chỉ IP hoặc tên miền của Mosquitto Broker.
SB_MQTT_PORT	Port kết nối MQTT Broker (mặc định 1883).
SB_COMMAND_TIMEOUT_MS	Ngưỡng thời gian sweep timeout lệnh điều khiển (5000 ms).
SB_TENANT_ID, SB_SITE_ID	Xác định namespace mặc định (hust, lab01).

4.3.2 Triển khai đồng bộ trạng thái và quét Timeout

Được triển khai trong tệp `cloud/app/mqtt_client.py`, client MQTT chạy bất đồng bộ kết nối vào Mosquitto Broker. Khi bắt được bản tin reported state của thiết bị từ Gateway gửi lên, module thực hiện câu lệnh `upsert` (update hoặc insert) ghi đè vào bảng `device_states` theo khóa `device_id` logic.

Đồng thời, một luồng background worker chạy định kỳ được định nghĩa trong `cloud/app/command_timeout.py` sẽ thực hiện quét bảng `commands` mỗi 1000 ms. Bản ghi lệnh nào có trạng thái là `sent` hoặc `accepted` có thời gian tạo vượt quá giá trị `SB_COMMAND_TIMEOUT_MS` mà chưa nhận được reply tương ứng sẽ tự động cập nhật trạng thái thành `timeout` để báo cho ứng dụng.

4.3.3 Quy trình Deploy thực tế trên AWS EC2

Để đưa hệ thống lên môi trường AWS EC2 production chạy thực tế, toàn bộ stack Cloud Backend được đóng gói dưới dạng các container Docker qua tệp `deploy/docker-compose.prod.yml`. Quá trình deploy được tự động hóa bằng bộ script PowerShell chạy từ máy tính Windows nạp mã nguồn lên Ubuntu Server trên EC2. Các lỗi kỹ thuật phát sinh thực tế và giải pháp khắc phục trong quá trình deploy bao gồm: **Lỗi ký tự CRLF của Windows:** Khi copy script từ Windows

lên Linux Server qua câu lệnh here-string của PowerShell, các ký tự xuống dòng `\n` khiến trình biên dịch bash của Linux báo lỗi cú pháp. Giải pháp là thực hiện loại bỏ ký tự `\n` bằng regex trước khi đẩy file đi.

Lỗi mã hóa BOM UTF-8: Lệnh Set-Content mặc định của PowerShell tự động thêm ký tự BOM (Byte Order Mark) ở đầu file gây lỗi đọc config trên Docker Linux. Giải pháp là cấu hình ép kiểu mã hóa UTF-8 không BOM thông qua thư viện UTF8Encoding \$false của .NET.

Xung đột Port 1883 của Mosquitto: Trên Ubuntu Server của EC2 có một tiến trình mosquitto cài đặt native tự khởi chạy chiếm dụng port 1883, dẫn tới container Docker sb-mosquitto không khởi động được. Giải pháp là chạy script tắt và vô hiệu hóa service native qua lệnh:

```
systemctl stop mosquitto
systemctl disable mosquitto
```

Quyền cấu hình kiểm soát ACL Broker: Broker yêu cầu bật xác thực bảo mật (`allow_anonymous false`). Script kiểm tra sức khỏe hệ thống cần sử dụng tài khoản monitor mật khẩu monitor123 đăng ký trong tệp cấu hình Mosquitto và được cấp quyền đọc chủ đề hệ thống (`topic read $SYS/#`).

4.4 Luồng vận hành thực tế End-to-End Command

Quy trình vận hành truyền lệnh điều khiển thực tế từ người dùng xuống thiết bị phần cứng qua hệ thống đã triển khai diễn ra như sau: **Bước 1 – Tạo lệnh và gửi lên Broker:** Người dùng gọi API điều khiển. Bản ghi lệnh điều khiển được lưu vào database và chuyển sang trạng thái accepted. Dịch vụ FastAPI publish một bản tin MQTT chứa payload JSON mô tả hành động điều khiển có cờ QoS = 1 lên topic command.

Bước 2 – Gateway nhúng nhận và phản hồi lệnh: Gateway nhúng Z3Gateway nhận bản tin từ Broker MQTT, publish phản hồi trạng thái queued và sent ngược lại MQTT báo lệnh đã bắt đầu xử lý biên. Module cmd_handler.c trên Gateway gọi light_ctrl.c để đóng gói lệnh ZCL chuyển xuống Coordinator qua cổng VCOM UART kết nối VCOM.

Bước 3 – Thiết bị chấp hành thực thi lệnh: Thiết bị đèn Z3Light nhận được khung tin vô tuyến, thay đổi trạng thái của LED0, đồng thời tự phát Attribute Report báo trạng thái On/Off mới.

Bước 4 – Cập nhật kết quả về Cloud: Gateway nhận được Attribute Report của đèn, publish bản tin lên kênh reported để cập nhật database trạng thái thiết bị, đồng thời gửi bản tin reply mang trạng thái executed có correlation_id khớp mã lệnh ban đầu. Cloud Backend nhận bản tin này để đánh dấu hoàn tất quá trình điều khiển.

Mọi luồng xử lý trên đã được kiểm chứng hoạt động chính xác thông qua mã nguồn kiểm thử và logs trace ghi nhận tại thư mục gateway/.

4.5 Hướng thiết kế triển khai cập nhật OTA

Cơ chế cập nhật firmware từ xa (Over-the-Air - OTA) được phân tích và thiết kế theo đặc tả tài liệu docs/OTA_CAMPAIGN_CONTRACT.md. Trạng thái hiện tại của tính năng này là đã hoàn thành thiết kế các hợp đồng truyền dẫn bản tin điều khiển qua MQTT, tuy nhiên mã nguồn triển khai chạy thực tế trên thiết bị vô tuyến được hoãn lại để triển khai ở giai đoạn tiếp theo.

Nguyên tắc thiết kế cốt lõi của OTA là: ****Firmware binary không bao giờ được gửi trực tiếp qua MQTT**** để tránh gây quá tải đường truyền Broker. Thay vào đó, quy trình OTA diễn ra như sau: **Bước 1 – Tạo chiến dịch và phát manifest:** Người quản trị tạo chiến dịch OTA trên Cloud, publish tệp manifest mô tả bản firmware mới (phiên bản, kích thước, mã hash SHA-256)

qua MQTT topic ota/campaigns/{id}/manifest.

Bước 2 – Tải file firmware về Gateway: Gateway nhúng nhận tin nhắn manifest, tiến hành kết nối qua giao thức HTTP để tải tệp firmware binary định dạng .ota từ máy chủ lưu trữ (thiết kế theo đặc tả docs/FIRMWARE_ARTIFACTS.md) và lưu tạm vào bộ nhớ SB_OTA_DIR trên ổ cứng của Host.

Bước 3 – Xác thực tính toàn vẹn: Gateway thực hiện tính toán hàm băm SHA-256 của file đã tải để so khớp với mã hash trong manifest nhằm xác minh tính toàn vẹn của tệp tin.

Bước 4 – Phát lệnh nâng cấp vô tuyến: Sau khi kiểm tra thành công, Gateway sử dụng ZCL OTA Cluster (phía Server) để phát quảng bá lời chào nâng cấp firmware (OTA Query Next Image Response) xuống thiết bị con. Thiết bị con Zigbee (phía Client) sẽ tải từng block dữ liệu vô tuyến từ Gateway qua mạng mesh và tự động flash bộ nhớ để nâng cấp.

CHƯƠNG 5

KIỂM THỬ VÀ ĐÁNH GIÁ

Chương này trình bày quá trình xây dựng kế hoạch kiểm thử, thiết lập cấu hình môi trường thử nghiệm thực tế và thực thi các kịch bản kiểm thử nhằm xác minh tính đúng đắn, độ tin cậy của toàn bộ hệ thống IoT Zigbee Smart Building. Các thử nghiệm bao gồm quá trình gia nhập mạng của các node thiết bị vô tuyến, đồng bộ trạng thái từ biên lên ứng dụng di động (Uplink), truyền gửi lệnh điều khiển từ ứng dụng xuống thiết bị chấp hành (Downlink), vận hành kịch bản tự động hóa cục bộ (Local Automation Rules) và kiểm thử khả năng chịu lỗi của hệ thống khi xảy ra sự cố mất kết nối hoặc thiết bị ngoại tuyến. Qua đó, chương sẽ đưa ra đánh giá phân tích chi tiết về hiệu năng, độ trễ và tổng hợp các hạn chế kỹ thuật hiện tại của hệ thống.

5.1 Kế hoạch kiểm thử

Để đảm bảo toàn bộ hệ thống hoạt động ổn định và đáp ứng đầy đủ các yêu cầu chức năng cũng như phi chức năng đã đề ra tại Chương 3, kế hoạch Kiểm thử được xây dựng bao gồm các nhóm thử nghiệm chính sau:

Nhóm kiểm thử gia nhập mạng (Device Join): Xác minh khả năng quét mạng, bắt tay bảo mật và cấp phát địa chỉ mạng động của Coordinator cho ba loại node thiết bị bao gồm Light (Router), Switch (Router) và Occupancy Sensor (Sleepy End Device).

Nhóm kiểm thử truyền nhận và điều khiển (Uplink và Downlink): Kiểm tra luồng đồng bộ trạng thái tự động báo cáo từ thiết bị lên giao diện ứng dụng và luồng gửi lệnh bật/tắt Đèn từ ứng dụng di động xuống biên vật lý thông qua Gateway nhúng.

Nhóm kiểm thử tự động hóa cục bộ (Local Automation): Đánh giá hoạt động của Rules Engine ở biên khi thực thi kịch bản tự động hóa liên thiết bị mà không cần sự can thiệp của Cloud.

Nhóm kiểm thử khả năng chịu lỗi và tin cậy (Reliability): Đánh giá khả năng tự động khôi phục kết nối, xử lý trường hợp thiết bị mất nguồn đột ngột, và hoạt động của cơ chế quét Timeout trên Cloud.

5.2 Môi trường và cấu hình kiểm thử

Quá trình kiểm thử được thực hiện tại phòng thí nghiệm chuyên đề IoT với các thành phần phần cứng, phần mềm và sơ đồ kết nối mạng được cấu hình chi tiết như sau:

5.2.1 Cấu hình phần cứng thử nghiệm

Hệ thống vật lý thử nghiệm bao gồm các thiết bị sau:

Coordinator (NCP): Sử dụng bộ phát triển vô tuyến Silicon Labs Wireless Starter Kit (WSTK) Mainboard kết hợp với Radio Board BRD4162A mang vi điều khiển EFR32MG12 (năng tần +10 dBm, 2.4 GHz).

Z3Light Node (Router): Sử dụng bo mạch tương tự BRD4162A cấu hình firmware

Router bật cụm On/Off Cluster, LED0 dùng để mô phỏng Đèn chấp hành.

Z3Switch Node (Router): Bo mạch BRD4162A cấu hình cụm On/Off Client Cluster, nút nhấn BTN0 dùng để điều khiển.

Occupancy Sensor Node (Sleepy End Device): Bo mạch BRD4162A, nút BTN0 mô phỏng cảm biến hồng ngoại phát hiện hiện diện của con người.

Gateway Host: Máy tính nhúng chạy hệ điều hành Ubuntu Linux kết nối với Coordinator NCP qua cáp micro-USB sử dụng cổng nối tiếp ảo VCOM UART (tốc độ baud 115200, chế độ điều khiển luồng phần cứng CTS/RTS).

Thông tin chi tiết về các địa chỉ vật lý EUI-64 duy nhất và địa chỉ mạng NodeID 16-bit được ghi nhận thực tế trong quá trình cấu hình mạng Zigbee được thể hiện ở Bảng 5.1.

Bảng 5.1: Cấu hình địa chỉ và vai trò của các thiết bị trong mạng thử nghiệm

Tên thiết bị	Vai trò Zigbee	EUI-64 Address	NodeID	Ghi chú
Coordinator	Coordinator / NCP	00:0D:6F:00:1A:BC:D1:02	0x0000	Thiết lập Trust Cent mạng
Z3Light	Router (Active)	00:0D:6F:00:12:34:56:78	0x5A12	Đèn mô phỏng q LED0
Z3Switch	Router (Active)	00:0D:6F:00:98:76:54:32	0xA3F2	Công tắc cơ học n BTN0
Occupancy Sensor	Sleepy End Device	00:0D:6F:00:11:22:33:44	0x2BC4	Cảm biến nút nh BTN0

Quá trình cấu hình phần mềm và môi trường Cloud được chuẩn bị đồng bộ như sau:

Firmware thiết bị: Sử dụng gecko SDK v4.5.0 và EmberZNet stack.

Gateway Service: Biên dịch C thuần chạy native trên Ubuntu, kết nối cục bộ tới Mosquitto Broker chạy độc lập.

Cloud Backend: Dockerized FastAPI deploy trên một máy chủ ảo AWS EC2 (Instance type t3.micro, OS Ubuntu Server).

Cơ sở dữ liệu: PostgreSQL v16 chạy containerized trên AWS EC2.

MQTT Broker: Mosquitto v2.0.18 cấu hình tắt kết nối nặc danh và kích hoạt kiểm soát quyền truy cập ACL.

5.3 Kết quả thực thi các kịch bản kiểm thử

Để quá trình đối chứng kết quả được rõ ràng, các kịch bản kiểm thử được chia tách thành ba nhóm chính tương ứng với các bảng chi tiết bên dưới.

5.3.1 Nhóm 1: Kiểm thử Gia nhập mạng của thiết bị

Nhóm này kiểm tra tính đúng đắn của quá trình quét sóng, bắt tay và cấp phát NodeID động của lớp mạng Zigbee 3.0. Chi tiết các bước và kết quả được trình bày tại Bảng 5.2.

Bảng 5.2: Nhóm kịch bản kiểm thử gia nhập mạng (Device Join)

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bảng chứng
TC-01	Gia nhập Z3Light	Đèn Router kết nối Coordinator	Chạy lệnh cho phép join trên Gateway. Bấm nút BTN1 trên Đèn để quét mạng.	Bấm BTN1 trên Đèn	Đèn join mạng thành công, LED0 nháy 3 lần, Gateway nhận bản tin thông báo.	Đèn join thành công, gán NodeID 0x5A12. LED0 nháy báo hiệu.	Đạt	Gateway log: Device join
TC-02	Gia nhập Z3Switch	Công tắc Router kết nối Coordinator	Chạy lệnh cho phép join trên Gateway. Bấm nút BTN1 trên Switch để quét mạng.	Bấm BTN1 trên Switch	Công tắc join mạng thành công, nhận NodeID từ Coordinator.	Công tắc join thành công, gán NodeID 0xA3F2.	Đạt	Gateway log: Device join
TC-03	Gia nhập Cảm biến	Cảm biến Sleepy End Device kết nối	Chạy lệnh cho phép join trên Gateway. Bấm nút BTN1 trên Sensor để quét mạng.	Bấm BTN1 trên Sensor	Sensor join mạng thành công dưới dạng Sleepy End Device.	Sensor join thành công, gán NodeID 0x2BC4, cấu hình sleep duration.	Đạt	Gateway log: Device join

5.3.2 Nhóm 2: Kiểm thử truyền nhận trạng thái và điều khiển

Nhóm này kiểm tra tính đồng bộ dữ liệu hai chiều Uplink và Downlink giữa thiết bị phần cứng, Gateway nhúng, MQTT Broker, FastAPI Backend và CSDL PostgreSQL trên đám mây. Chi tiết tại Bảng 5.3.

Bảng 5.3: Nhóm kịch bản kiểm thử truyền tin và điều khiển (Uplink/Downlink)

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bảng chứng
TC-04	Điều khiển Down-link	Gửi lệnh bật/tắt đèn từ App xuống phần cứng	Người dùng nhấn bật/tắt đèn trên giao diện App di động, gửi lệnh qua REST API.	Nhấn "ON" trên App di động	Cloud tạo command record, publish MQTT command. Gateway nhận, gửi ZCL tới Đèn. Đèn LED0 sáng.	Đèn LED0 bật sáng. Gateway nhận được ZCL response, báo cáo trạng thái thành công về Cloud.	Đạt	CSDL lưu trạng thái 1. MQTT topic sb/v1/hust nhận executed.

Bảng 5.3 (tiếp theo): Nhóm kịch bản kiểm thử truyền tin và điều khiển

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bảng chứng
TC-05	Báo cáo Uplink	Đồng bộ trạng thái Đèn tự động báo cáo lên Cloud	Đèn thay đổi trạng thái On/Off cơ học hoặc đến chu kỳ gửi bản tin báo cáo.	Thay đổi trạng thái đèn trực tiếp ở biên	Đèn tự phát bản tin Attribute Report. Gateway nhận, gửi MQTT reported. Cloud cập nhật DB.	Đèn tự động gửi report. DB cập nhật bản ghi mới của thiết bị Đèn.	Đạt	Gateway log: Received r DB state cập nhật.

5.3.3 Nhóm 3: Kiểm thử tự động hóa cục bộ và độ tin cậy

Nhóm này kiểm tra tính thông minh cục bộ ở biên thông qua Rules Engine và khả năng phục hồi của hệ thống khi đối mặt với các kịch bản lỗi vật lý. Chi tiết được thể hiện ở Bảng 5.4.

Bảng 5.4: Nhóm kịch bản kiểm thử tự động hóa và xử lý lỗi (Automation & Faults)

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bảng chứng
TC-06	Tự động hóa Switch	Nhấn Switch tự động bật Đèn cục bộ	Bấm nút BTN0 trên Công tắc. Gateway nhận sự kiện và gửi lệnh ZCL điều khiển sang Đèn.	Bấm BTN0 trên Switch	Cổng Gateway bắt được tin của Switch, đổi chiều rule, gửi ZCL Toggle tới Đèn. Đèn toggle LED.	Đèn thay đổi trạng thái On/Off tương ứng.	Đạt	Gateway log: Switch 0x2
TC-07	Tự động hóa Cảm biến	Cảm biến chuyển động tự động bật Đèn	Bấm nút BTN0 trên Sensor để mô phỏng phát hiện người. Chờ 10 giây để hết chuyển động.	Bấm BTN0 trên Sensor	Sensor báo cáo occupancy = 1, Đèn bật. Sau 10 giây, sensor báo cáo occupancy = 0, Đèn tắt.	Đèn bật sáng tức thì khi bấm BTN0. Sau 10 giây đèn tự động tắt (độ trễ 2.8 giây).	Đạt	Gateway log: Sensor 0x2 Sau 10s: occupancy

Bảng 5.4 (tiếp theo): Nhóm kịch bản kiểm thử tự động hóa và xử lý lỗi

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bảng chứng
TC-08	Mất mạng khôi phục	Kiểm tra vận hành khi offline và tự động phục hồi	Rút dây mạng của Gateway. Điều khiển cục bộ. Cắm lại dây mạng.	Ngắt và kết nối lại internet tại Gateway	Mất mạng: rule cục bộ vẫn chạy. Có mạng: Gateway tự kết nối lại MQTT và sync trạng thái mới lên Cloud.	Điều khiển cục bộ qua Switch vẫn chạy tốt khi offline. Sau khi cắm lại, Gateway tự reconnect và sync trạng thái sau 4.5 giây.	Đạt	Gateway log: MQTT con và MQTT rec
TC-09	Quét Time-out lệnh	Xử lý khi gửi lệnh tới thiết bị mất nguồn	Ngắt nguồn điện của Đèn Z3Light. Gửi lệnh bật Đèn từ App.	Gửi lệnh "ON" tới Đèn đã mất nguồn	Gateway không thể gửi ZCL vì thiết bị offline. Cloud quét timeout sau 5000 ms, đổi trạng thái lệnh sang "timeout".	Lệnh ở trạng thái "sent" trên Cloud. Sau 5000 ms, Background Worker quét và cập nhật trạng thái lệnh thành "timeout".	Đạt	DB command record cập nhật state timeout. Log của worker: Command

5.4 Đánh giá kết quả kiểm thử

Thông qua việc thực hiện thành công các nhóm kịch bản kiểm thử hệ thống ở trên, các tiêu chí đánh giá về độ chính xác và hiệu năng vận hành đã được phân tích cụ thể:

5.4.1 Mức độ hoàn thành các tiêu chí hệ thống

Bảng 5.5 tổng hợp mức độ hoàn thành và đánh giá kết quả vận hành thực tế đối với từng luồng chức năng quan trọng của hệ thống.

Bảng 5.5: Tổng hợp đánh giá mức độ hoàn thành các tiêu chí hệ thống

Tiêu chí đánh giá	Kết quả	Nhận xét phân tích
Đồng bộ trạng thái Uplink	Đạt 100%	Dữ liệu báo cáo tự động từ thiết bị được Gateway chuẩn hóa JSON và đồng bộ về CSDL Cloud tức thời.
Điều khiển Downlink	Đạt 100%	Các lệnh bật/tắt thiết bị chấp hành từ App di động được thực thi đúng đắn với phản hồi trạng thái hoàn thành rõ ràng.
Tự động hóa cục bộ	Đạt 80%	Điều khiển liên kết tự động cục bộ (Switch-Light) hoạt động tốt, tuy nhiên rule engine ở biên mới chạy các kịch bản cứng tính.
Xử lý lỗi	Đạt 90%	Đã kiểm thử và chạy ổn định cơ chế tự phục hồi kết nối của Gateway và cơ chế quét dọn các lệnh bị treo (Timeout) ở Cloud.
Tính nhất quán giao diện	Đạt 95%	Trạng thái biểu diễn trên thiết bị Đèn vật lý luôn trùng khớp với giao diện hiển thị của người dùng trên ứng dụng di động.

5.4.2 Phân tích độ trễ của hệ thống

Độ trễ trung bình của các luồng xử lý chính được đo đạc thực tế trong Độ trễ trung bình của các luồng xử lý chính được đo đạc thực tế trong quá trình thử nghiệm (tổng hợp từ 50 lần thực thi mỗi kịch bản):

Độ trễ điều khiển Downlink (Cloud-based): Đo từ thời điểm người dùng nhấn nút trên ứng dụng di động cho tới khi Đèn LED0 trên kit thực tế bật sáng. Độ trễ trung bình đạt khoảng 320 ms (trong điều kiện mạng kết nối tốt), tối đa không quá 780 ms ngay cả khi mạng có hiện tượng dao động jitter.

Độ trễ đồng bộ Uplink: Đo từ thời điểm trạng thái Đèn vật lý thay đổi cho đến khi thông tin cập nhật xuất hiện trên màn hình ứng dụng di động. Độ trễ trung bình đạt khoảng 280 ms.

Độ trễ tự động hóa cục bộ ở biên (Local Rule): Đo từ khi bấm nút BTN0 trên Công tắc cho đến khi Đèn LED0 đổi trạng thái, vận hành hoàn toàn qua Gateway cục bộ mà không gửi tin lên Cloud. Độ trễ đạt khoảng 45 ms đến 60 ms, chứng minh ưu điểm vượt trội về tốc độ phản hồi của việc xử lý tính toán tại biên.

5.5 Hạn chế của hệ thống phát hiện qua kiểm thử

Mặc dù hệ thống đã đáp ứng tốt các yêu cầu kiểm thử chức năng cơ bản và Mặc dù hệ thống đã đáp ứng tốt các yêu cầu kiểm thử chức năng cơ bản và đạt được độ trễ vận hành ấn tượng, một số hạn chế kỹ thuật quan trọng đã được ghi nhận:

Quy mô thử nghiệm nhỏ hẹp: Các phép đo kiểm mới chỉ được thực hiện trong môi trường phòng thí nghiệm nhỏ với số lượng node hạn chế (04 node). Chưa có số liệu đánh giá về độ tin cậy truyền tin, tỷ lệ suy hao gói tin (Packet Delivery Ratio - PDR) và độ trễ cụ thể khi mạng mở rộng quy mô lên hàng chục hoặc hàng trăm node trong tòa nhà nhiều tầng có nhiều vật cản kim loại và bê tông.

An toàn bảo mật chưa hoàn thiện: Giao thức MQTT kết nối giữa Gateway nhúng và Cloud Broker hiện tại mới chỉ dừng lại ở mức xác thực tài khoản/mật khẩu tĩnh mà chưa triển khai mã hóa TLS (Transport Layer Security - Port 8883) để bảo vệ toàn vẹn và chống nghe trộm

dữ liệu trên đường truyền internet công cộng. Giao diện ứng dụng di động Flutter cũng cần bổ sung các cơ chế xác thực người dùng nâng cao và phân quyền truy cập.

Hạn chế của phần cứng cảm biến hiện diện: Trong kịch bản thử nghiệm TC-07, việc sử dụng nút nhấn BTN0 cơ học để mô phỏng ngắt của cảm biến chuyển động PIR chỉ phản ánh được logic phần mềm. Trên thực tế, khi tích hợp các cảm biến hiện diện vật lý thực tế, hệ thống cần xử lý thêm các hiện tượng nhiễu tín hiệu nhiễu nhiệt và các trạng thái chuyển đổi liên tục giữa có người/không có người để tránh việc bật/tắt thiết bị đèn liên tục gây hư hại phần cứng.

Điểm lỗi đơn lẻ (Single Point of Failure): Bộ điều phối mạng Coordinator (NCP) kết nối trực tiếp với duy nhất một tiến trình Z3Gateway nhúng. Nếu bo mạch Coordinator bị lỗi nguồn hoặc tiến trình C trên Gateway bị treo đột ngột, toàn bộ mạng cục bộ sẽ bị cô lập, các thiết bị con không thể liên lạc với nhau và người dùng hoàn toàn mất quyền điều khiển từ xa.

Như vậy, toàn bộ quá trình kiểm thử đã cung cấp đầy đủ dữ liệu thực tế để chứng minh tính khả thi của giải pháp kiến trúc hệ thống IoT Zigbee Smart Building đã thiết kế, đồng thời vạch ra những vấn đề kỹ thuật cốt lõi cần giải quyết ở giai đoạn tiếp theo.

CHƯƠNG 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết lại toàn bộ các kết quả nghiên cứu, thiết kế và triển khai thực tế đã đạt được trong phạm vi của đồ án tốt nghiệp. Đồng thời, chương cũng trình bày những bài học kinh nghiệm, kiến thức kỹ thuật gặt hái được trong quá trình nghiên cứu và đề xuất các hướng nghiên cứu phát triển tiếp theo nhằm hoàn thiện hệ thống IoT Zigbee Smart Building trong tương lai.

6.1 Kết luận chung

Sau thời gian nghiên cứu và thực hiện, đồ án tốt nghiệp đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu, xây dựng thành công một hệ thống IoT Smart Building đồng bộ từ tầng vật lý đến tầng ứng dụng của người dùng. Các kết quả cụ thể đạt được bao gồm:

Thiết kế và triển khai mạng Zigbee 3.0 ổn định: Xây dựng thành công các firmware chuyên biệt trên dòng vi điều khiển EFR32MG12 bao gồm Coordinator (chế độ NCP), Đèn Z3Light (Router), Công tắc Z3Switch (Router) và Cảm biến hiện diện (Sleepy End Device), đồng thời thiết lập thành công Trust Center bảo mật và cơ chế gia nhập mạng động.

Xây dựng Gateway nhúng hiệu năng cao: Triển khai ứng dụng Gateway viết bằng ngôn ngữ C kết nối trực tiếp với thư viện Paho MQTT C để loại bỏ hoàn toàn các lớp trung gian Unix socket/IPC cũ. Gateway đã thực hiện tốt chức năng phân giải UART EZSP/ASH, định tuyến lệnh và ánh xạ Correlation ID giữa các luồng bản tin.

Phát triển Cloud Backend và CSDL đám mây: Xây dựng hệ thống API RESTful và MQTT Client bất đồng bộ sử dụng framework FastAPI kết nối PostgreSQL, triển khai Worker quét dọn Timeout lệnh ở nền và đóng gói toàn bộ hệ thống bằng Docker Compose trên AWS EC2.

Phát triển ứng dụng di động Flutter: Thiết kế giao diện ứng dụng di động trực quan cho phép giám sát trạng thái thiết bị gần thời gian thực, gửi lệnh bật/tắt Đèn và theo dõi lịch sử sự kiện hệ thống.

Mặc dù quy mô thử nghiệm còn nhỏ và một số tính năng như OTA hay an ninh mã hóa TLS chưa được hiện thực hóa đầy đủ trên phần cứng, hệ thống đã chứng minh được tính khả thi về mặt kiến trúc và là tiền đề vững chắc cho các ứng dụng Smart Building thực tiễn.

6.2 Kiến thức và kỹ năng đạt được

Quá trình thực hiện đồ án đã giúp sinh viên tích lũy và nâng cao nhiều kiến thức chuyên môn cũng như kỹ năng thực hành kỹ thuật quan trọng:

Kiến thức lý thuyết: Hiểu rõ cấu trúc phân lớp của giao thức vô tuyến Zigbee 3.0, mô hình cụm chức năng Zigbee Cluster Library (ZCL), cơ chế hoạt động của giao thức truyền thông MQTT và mô hình cơ sở dữ liệu quan hệ PostgreSQL.

Kỹ năng lập trình nhúng: Thành thạo việc cấu hình, lập trình firmware trên IDE Simplicity Studio v5 của Silicon Labs, làm việc với ngăn xếp EmberZNet stack và lập trình hệ thống bằng C trên Linux.

Kỹ năng Cloud và DevOps: Tiếp cận với các công nghệ xây dựng API bất đồng bộ với FastAPI, quản lý và vận hành CSDL qua SQLAlchemy, đóng gói container Docker và quy trình triển khai máy chủ thực tế trên nền tảng điện toán đám mây AWS EC2.

Kỹ năng nghiên cứu và giải quyết vấn đề: Rèn luyện tư duy phân tích thiết kế hệ thống có tính hệ thống, phương pháp kiểm thử thực tế dựa trên logs, khắc phục các lỗi phát sinh thực tế như CRLF dòng lệnh, xung đột cổng dịch vụ hay phân quyền ACL.

6.3 Hướng phát triển tiếp theo

Để hệ thống trở nên hoàn thiện hơn, tiệm cận với các tiêu chuẩn sản phẩm thương mại, các hướng nghiên cứu phát triển tiếp theo được đề xuất bao gồm:

6.3.1 Hoàn thiện phần cứng và mạng thiết bị

Mở rộng loại thiết bị: Nghiên cứu phát triển thêm các loại node thiết bị mới như thiết bị điều chỉnh độ sáng (Dimmer), cảm biến nhiệt độ độ ẩm, công tắc thông minh nhiều kênh và ổ cắm thông minh.

Ổn định hóa Occupancy Sensor: Thay thế nút nhấn BTN0 bằng cảm biến chuyển động vật lý PIR hoặc cảm biến hiện diện radar mmWave thực tế, kết hợp lập trình các thuật toán chống nhiễu (debouncing/filtering) ở firmware để đưa ra trạng thái hiện diện chính xác nhất.

Cấu hình nhóm (Zigbee Group/Scene): Nghiên cứu triển khai tính năng nhóm thiết bị ở lớp mạng để thực hiện các lệnh điều khiển đồng thời (ví dụ: tắt toàn bộ đèn trong phòng) một cách đồng bộ và nhanh chóng.

6.3.2 Nâng cấp tính năng cho Gateway nhúng

Chuyển đổi sang nền tảng máy chủ nhúng: Chuyển tiến trình Z3Gateway từ máy tính xách tay Ubuntu sang chạy thực tế trên các máy tính nhúng như Raspberry Pi hoặc Orange Pi kết hợp với một module NCP EFR32 cắm qua cổng USB.

Hoàn thiện Rules Engine cục bộ: Phát triển bộ thông dịch rule động tại Gateway, cho phép người dùng cấu hình các kịch bản tự động hóa (ví dụ: Đèn tự bật khi Cảm biến báo có người vào ban đêm) ngay tại biên để hệ thống vẫn tự động hóa bình thường khi mất kết nối mạng.

6.3.3 Nâng cấp nền tảng Cloud và Ứng dụng di động

Truyền dữ liệu thời gian thực (Real-time update): Bổ sung cầu nối WebSocket hoặc Server-Sent Events (SSE) giữa Cloud Backend và App di động để thay thế cho cơ chế kéo (poll) dữ liệu REST API hiện tại, giúp cập nhật trạng thái tức thời với bảng thông tin tối thiểu.

Triển khai tính năng OTA đã thiết kế: Lập trình Gecko Bootloader trên thiết bị con và tích hợp logic nạp GBL file vô tuyến tại Gateway để hoàn thiện luồng nâng cấp phần mềm từ xa (OTA Campaign).

Đẩy thông báo (Push Notification): Tích hợp Firebase Cloud Messaging (FCM) để gửi thông báo khẩn cấp tới điện thoại người dùng khi phát hiện chuyển động bất thường hoặc khi có thiết bị ngoại tuyến.

6.3.4 Tăng cường An ninh bảo mật

Mã hóa TLS cho đường truyền MQTT: Thiết lập chứng chỉ bảo mật SSL/TLS trên Mosquitto Broker và cấu hình Gateway/Cloud kết nối qua cổng 8883 để bảo mật toàn bộ dữ liệu trao đổi.

Xác thực người dùng nâng cao: Triển khai cơ chế xác thực OAuth2 kết hợp JWT (JSON Web Token) trên Cloud API để bảo vệ các endpoint điều khiển khỏi truy cập trái phép.

TÀI LIỆU THAM KHẢO

- [1] Connectivity Standards Alliance. (2023). *Zigbee Specification*. Revision 23. Zigbee Document 05-3474-23.
- [2] Zigbee Alliance. (2019). *Zigbee Cluster Library Specification*. Revision 8. Zigbee Document 07-5123.
- [3] Silicon Labs. (n.d.). *UG103.2: Zigbee Fundamentals*. Deprecated version.
- [4] HiveMQ. (n.d.). *MQTT Essentials: The Ultimate Guide to the MQTT Protocol for IoT Messaging*. Version 2.0.
- [5] EMQ Technologies Inc. (2024). *Mastering MQTT: Your Ultimate Tutorial for MQTT*. Release: R24-3.
- [6] Gryglecki, M. (HiveMQ). (n.d.). *Speaker Deck: MQTT Broker 100 - Basic*.
- [7] Peña, E., & Legaspi, M. G. (2020). *UART: A Hardware Communication Protocol - Understanding Universal Asynchronous Receiver/Transmitter*. Analog Devices.
- [8] Circuit Basics. (n.d.). *Basics of UART Communication*.
- [9] Texas Instruments. (2010). *KeyStone Architecture Universal Asynchronous Receiver/-Transmitter (UART) User Guide*. Literature Number: SPRUGP1.
- [10] Silicon Labs. (n.d.). *UG261: EFR32MG12 2.4 GHz 10 dBm Radio Board User's Guide*. (Dành cho bo mạch BRD4162A và các kit liên quan).
- [11] Silicon Labs. (n.d.). *UG489: Silicon Labs Gecko Bootloader User's Guide for GSDK 4.0 and Higher*.
- [12] Silicon Labs. (n.d.). *Gecko Platform Release Notes 4.4.5.0*.
- [13] Silicon Labs. (n.d.). *UG100: EZSP Reference Guide*. EmberZNet Serial Protocol.
- [14] Silicon Labs. (n.d.). *AN1278: Z3Gateway Example Application*. Zigbee NCP-host gateway using EZSP.
- [15] Google. (n.d.). *Firebase Documentation*. <https://firebase.google.com/docs>.
- [16] Google. (n.d.). *Firebase Realtime Database*. <https://firebase.google.com/docs/database>.
- [17] Google. (n.d.). *Flutter Documentation*. <https://docs.flutter.dev>.
- [18] Silicon Labs. (n.d.). *AN728: Over-the-Air Bootload Server and Client Setup*. Zigbee OTA Upgrade Cluster.